Published: 10 March 2026
Accepted: 02 March 2026
Revised: 27 February 2026
Received: 12 August 2025

RESEARCH-ARTICLE

An Empirical Study on the Code Refactoring Capability of Large Language Models

JONATHAN CORDEIRO

SHAYAN NOEI

YING ZOU

An Empirical Study on the Code Refactoring Capability of Large Language Models

JONATHAN CORDEIRO, Queen's University, Canada
SHAYAN NOEI, Queen's University, Canada
YING ZOU, Queen's University, Canada

Large Language Models (LLMs) aim to generate and understand human-like text by leveraging deep learning and natural language processing techniques. In software development, LLMs can enhance the coding experience through coding automation, reducing development time and improving code quality. Code refactoring is a technique used to enhance the internal quality of the code base without altering its external functionalities. Leveraging LLMs for code refactoring can help developers improve code quality with minimal effort. This paper presents an empirical study evaluating the quality of refactored code produced by StarCoder2, GPT-4o-mini, GPT-4o, LLaMA 3, and DeepSeek-v3. Specifically, we (1) evaluate whether the code refactored by the LLMs can improve code quality, (2) understand the differences between the types of refactoring applied by the different LLMs and compare their effectiveness, and (3) evaluate whether the quality of the refactored code generated by the LLM can be improved through one-shot prompting and chain-of-thought prompting. We analyze the refactoring capabilities of LLMs on 30 open-source Java projects. We evaluate StarCoder2, LLaMA 3, GPT-4o-mini, GPT-4o, and DeepSeek-v3 on their ability to improve static code quality metrics, pass unit tests, and reduce code smells. Our findings reveal that production-grade models such as GPT-4o and DeepSeek-v3 achieve pass@5 unit test success rates above 90% on multi-file refactorings. LLaMA 3 achieves the highest overall code smell reduction with a median reduction of 15.1%, while DeepSeek-v3 and GPT-4o achieve the greatest improvements in cohesion, coupling, and complexity. StarCoder2 demonstrates strengths in modularity improvements and systematic refactorings. Developers outperform LLMs in complex, context-sensitive refactorings such as attribute encapsulation. We also show that prompt engineering significantly affects LLM performance: chain-of-thought prompting improves StarCoder2's test pass rate by 1.7% and increases code smell reduction compared to zero-shot prompting. One-shot prompting also expands the variety of refactorings LLMs can perform. These results suggest that LLMs are effective for many refactoring tasks, especially when guided with tailored prompts, but benefit from integration with human expertise for architectural or semantically complex changes. By providing insights into the capabilities and best practices for integrating LLMs into the software development process, our study aims to enhance the effectiveness and efficiency of code refactoring in real-world applications.

CCS Concepts: • **Software and its engineering**;

Additional Key Words and Phrases: Code Refactoring, Artificial Intelligence, Large Language Models, Auto-Generated Code, Code Quality

1 Introduction

Code refactoring improves internal code quality without changing its external behavior [23, 24, 48]. Automatic code refactoring allows developers to refactor code more efficiently and consistently [46]. However, developers have contextual knowledge and experience in refactoring the codebase, which is challenging for automated

Authors' Contact Information: Jonathan Cordeiro, 19jac16@queensu.ca, Queen's University, Kingston, Canada; Shayan Noei, s.noei@queensu.ca, Queen's University, Kingston, Canada; Ying Zou, ying.zou@queensu.ca, Queen's University, Kingston, Canada.

ACM 1557-7392/2026/3-ART
https://doi.org/10.1145/3801158

systems to replicate [23, 24, 79]. In recent years, Large Language Models (LLMs) have demonstrated capabilities in automatic code generation and in solving programming problems [21, 81]. In addition to these advances, LLMs come with certain limitations. Since LLMs are trained on large-scale corpora of programming source code, flaws in the training data can lead to the generation of poorly designed code [85], introduce technical debts [12], or cause hallucinated functionality [39]. Therefore, integration of the generated code into the software development process may increase maintenance costs and reduce the overall quality of the software [85].

Prior work has explored the use of LLMs for automating code refactoring [15, 69]. Most existing studies evaluate refactorings in constrained settings, often focusing on single-file or method-level refactorings, and primarily using zero-shot prompting, which leaves open questions about how LLMs perform in more realistic, multi-file refactoring scenarios. While some recent studies have begun to consider refactorings that span multiple files [11, 58], they typically focus on specific refactoring types or operate under restricted experimental settings, rather than evaluating commit-level transformations. In our dataset, we observe that 57.84% of refactoring commits modify more than one file, indicating that multi-file changes are the norm rather than the exception in practice. This motivates the need for commit-level, multi-file evaluation of LLM-based refactoring covering more levels of granularity. Accordingly, our work evaluates LLMs on refactorings at the commit level, where multiple files must be transformed consistently, and across a broad range of refactoring types and projects, providing a more realistic and comprehensive assessment of LLM-based refactoring.

To better understand how LLMs handle refactoring tasks, we conduct an empirical study on 30 open-source Java projects with 5,194 refactoring commits. These commits consist solely of refactoring operations within the common refactoring categories described in the literature [24] that can be detected by RMiner [77, 78], making them suitable baseline candidates representing developer-performed refactorings codes. We evaluate the effectiveness of LLMs in performing refactoring tasks and provide an evaluation framework to systematically assess the quality and reliability of LLM-generated refactorings across diverse real-world scenarios. To assess the quality of the generated refactorings, we use code quality metrics along with code smells at the implementation level, design level, testability level, and architectural levels. Our analysis includes refactoring types detectable by RMiner, which represent the current state of practice in automated refactoring detection. This procedure ensures that the code transformations targeting other software qualities, such as long-term product extensibility or architectural evolution beyond standard refactoring practices, are excluded from our dataset. Finally, given the wide range of refactoring types developers can perform, we analyze the coverage of refactoring types applied by LLMs relative to those observed in developers' commit history.

To perform our experiments, we utilize 5 popular LLMs, namely StarCoder2, LLaMA 3, DeepSeek-v3, GPT-4o, and GPT-4o-mini for our experiments. We first extract the code at the file level before a developer's refactoring and the code after the developer's refactoring. Moreover, using various prompting techniques (i.e., zero-shot, one-shot, and chain-of-thought), we supply each model with code blocks spanning multiple related files, enabling them to account for inter-file dependencies, data flows, and control flows. The LLMs are then tasked with performing the corresponding refactorings.

We evaluate the refactorings generated by LLMs using: (1) test cases, to ensure that the refactored code preserves the original functionality; and (2) a set of quality metrics, to assess whether the LLM-generated refactorings improve code quality and reduce code smells. Our work evaluates the LLMs' capability to perform all refactoring types. Importantly, we do not specifically instruct the LLMs to perform a particular refactoring type, allowing us to observe and assess their ability to autonomously choose and apply appropriate refactoring actions across diverse scenarios.

We aim to answer the following four research questions:

RQ1: What is the performance of LLMs in code refactoring? We aim to assess whether the LLMs can be used as reliable solutions to automate code refactoring. We evaluate their effectiveness with the reduction of code smells and their improvement on code quality, measured by various code metrics. We observe that, on

average, StarCoder2 achieves a 6.48% reduction in code smells, LLaMA 3 achieves a 15.73% reduction, GPT-4o Mini achieves a 7.89% reduction, DeepSeek-v3 achieves an average 11.05% reduction, and GPT-4o achieves a 12.84% reduction.

RQ2: Which quality aspects of the code are most effectively reduced by LLMs? To evaluate the ability of LLMs to improve code quality through refactoring, we identify the types of code smells that can be efficiently removed. LLaMA 3 excels in improving all code metrics analyzed, often surpassing StarCoder2 and GPT-4o Mini in these areas. DeepSeek-v3 and GPT-4o further extend this advantage, achieving the greatest overall improvements in cohesion, coupling, and complexity metrics. We observe that StarCoder2 excels in reducing structural and readability issues, while GPT-4o Mini is more effective at addressing design and abstraction challenges in code quality. DeepSeek-v3 and GPT-4o provide the most balanced and comprehensive improvements across all metric categories.

RQ3: Which refactoring types are most effective for improving code quality? To understand the capabilities of LLMs in handling different types of refactorings, we compare the refactoring types that it performs to effectively eliminate code smells and improve code quality with those performed by developers. Our findings show that StarCoder2 and GPT-4 excel at *Add Attribute Annotation*. In terms of code metrics improvement, developers are more effective at complex, context-sensitive refactorings such as *Rename Class* and *Encapsulate Attribute*, whereas LLaMA3 outperforms other LLMs in specific transformation tasks like *Change Return Type* and *Move Method*.

RQ4: How does prompt engineering affect the quality of LLM-generated refactorings? To investigate the impact of prompt engineering on refactoring quality, we evaluated zero-shot, one-shot, and chain-of-thought prompting techniques across three LLMs (StarCoder2, LLaMA 3, and GPT-4). Our results indicate that chain-of-thought prompting consistently yields the best performance for StarCoder2 and GPT-4, improving unit test pass rates to 35.1% and 80.1%, respectively, while also significantly increasing their code smell reduction rates (SRR). Specifically, StarCoder2's SRR improved to 7.05%, and GPT-4's SRR reached 7.20%, underscoring the effectiveness of structured reasoning in enhancing refactoring performance. For LLaMA 3, however, zero-shot prompting already provides strong baseline performance (73.4% pass rate and 15.82% SRR), and additional prompting techniques offer minimal further improvement.

Our work makes the following main contributions:

- We conduct a comprehensive evaluation of the capabilities of LLMs in automated code refactoring.
- We develop an evaluation framework for measuring the effectiveness of LLMs in code refactoring, with a focus on code quality improvement and functionality preservation.
- We compare different prompting techniques, including chain-of-thought and one-shot prompting, to assess their impact on the generation of high-quality refactorings and offer practical guidance on their application.
- We provide a replication package to enable the reproducibility of our study. The replication package of the study can be accessed at: https://github.com/Software-Evolution-Analytics-Lab-SEAL/LLM_Refactoring_Evaluation

Paper Organization. The remainder of our study is organized as follows. Section 3 describes the experiment setup of this study. Section 4 presents the motivation, approaches, and results of our research questions. Section 5 discusses the threats to the validity of our findings. Section 6 surveys related studies and compares them to our work. Finally, we conclude our paper and present future research directions in Section 7.

2 Background

Refactoring aims to improve the internal quality of software without changing its functionalities [23, 48]. Refactoring includes different types that each enhance a certain scope of the code, such as renaming variables, extracting

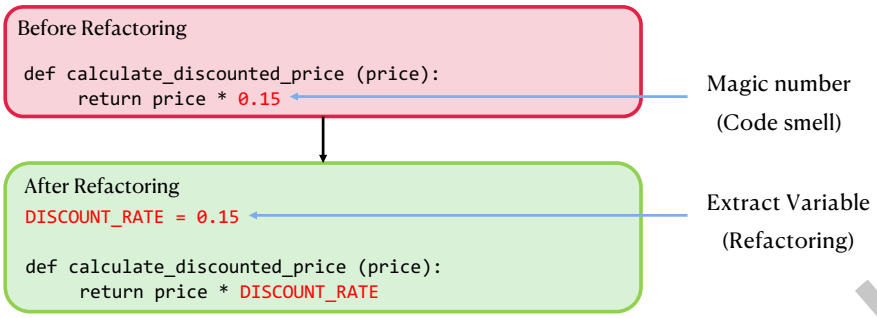

Fig. 1. Refactoring example demonstrating the removal of a *Magic Number* code smell by applying the *Extract Variable* refactoring type.

methods, and moving classes or methods [27]. Refactoring can improve software design, make code easier to understand, and remove dependencies among software components [44, 48, 51]. Refactoring is typically associated with the elimination of code smells [53]. Code smells are symptoms of poor design or implementation that negatively affect maintainability and evolution [53, 79]. These code smells span multiple categories, including architecture (e.g., unstable dependencies), design (e.g., unnecessary abstraction), implementation (e.g., long methods and duplicated code), and testability smells(e.g., excessive dependencies). The removal of code smells is commonly used as a measurable indicator of improved internal code quality after refactoring [1, 17, 24]. An example of refactoring is shown in Figure 1.

Previous studies have explored the potential of LLMs (e.g., GPT-3 [69]) in automating code refactoring, where LLMs are provided with a prompt and the code to be refactored [15, 69]. Furthermore, early static code generation methods mainly rely on rule-based systems or program synthesis approaches with limited flexibility. Program synthesis approaches construct code by exploring a space of candidate programs defined by formal grammars, symbolic constraints, or example-based specifications. While these techniques can guarantee correctness under certain conditions, they often struggle with scalability and flexibility when applied to complex real-world software. For example, DeepCoder [3] synthesizes small-scale programs from input-output examples, but struggles with more complex tasks. These approaches depend heavily on formal rules and lack the capacity to generalize effectively across diverse programming problems. Refactoring requires preserving the original program's functional behavior while restructuring and improving its design, a task that demands both syntactic and semantic understanding of the code. In contrast, program synthesis tools are typically designed to generate new programs from scratch based on a specification, rather than to transform and improve existing code. While prior work has advanced LLM-based refactoring, several challenges remain unexplored:

(1) Most LLM-driven code refactoring studies focus on single-file or method-level transformations [69]. While some recent work [11] has begun to consider refactorings that span multiple files, these studies focus on specific refactoring types or scoped experimental settings. As a result, broad, commit-level evaluation of multi-file refactorings remains underexplored, even though many real-world refactorings span multiple files or commits (e.g., *Pull Up Method* or *Extract Superclass*).
(2) Existing work [14, 37, 69] emphasizes syntactic correctness and optimizing surface-level metrics (e.g., reducing lines of code and lowering cyclomatic complexity), and does not explicitly model dependencies across files. However, such dependencies are critical in many refactoring scenarios, as understanding

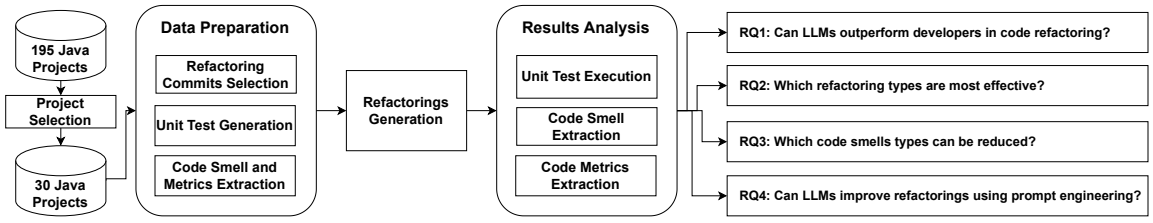

Fig. 2. Overview of Our Approach for Data Collection and Answering Research Questions.

control flows, data flows, and program logic among files is necessary to ensure that the LLM-proposed changes do not break functionality.

(3) Higher-level code quality objectives, such as modularity (e.g., the degree to which code components are decoupled and reusable) and readability, receive comparatively less attention in prior work [5, 57, 58, 69]. Such quality objectives are vital for long-term code sustainability. For example, Shirafuji et al. [69] evaluate LLM-driven code refactoring primarily through test-based repair outcomes or improvements in code metrics (e.g., lines of code and cyclomatic complexity), without directly linking to the higher-level quality objectives.

(4) Although strategies like few-shot and chain-of-thought (CoT) prompting have shown promise in enhancing model reasoning and code correctness (as observed in benchmarks like HumanEval and MBPP) [9], their potential to improve refactoring in realistic, commit-level settings has not yet been extensively explored.

(5) Prior studies [58, 69] typically evaluate LLMs on a limited set of refactoring types, primarily simple function-level transformations such as *Rename Method*, *Extract Method*, and *Inline Method*.

(6) Most existing evaluations rely on closed-source LLMs with non-transparent training corpora, making it difficult to assess the extent to which reported performance may be influenced by training data leakage [2, 18].

To address these gaps, our approach integrates refactorings at the commit level by prompting the LLM to simultaneously refactor all files involved in a change. Our work aims to conduct a large-scale evaluation covering a wide range of refactoring types across different granularities, including class, method, and variable level refactorings.

3 Experiment Setup

This section describes the experimental setup of our study, including project selection, data preparation, and analysis methods.

3.1 Overview of Our Approach

Figure 2 shows the overview of our study. We conduct our experiments by systematically selecting 30 projects from our initial dataset. On the selected projects, we prepare data by extracting the refactoring commits (i.e., commits in which developers exclusively performed refactoring) along with their associated files, code smells, and code metrics. Then, using different prompting techniques, we prompt LLMs to generate refactorings on the extracted files from the selected commits before the developers' refactoring operations. Furthermore, we analyze the refactored code generated by LLMs and compare the refactored code across several dimensions: the types of refactorings performed, the effectiveness of different prompting strategies, improvements in code quality metrics and code smell reduction, and functional correctness in the form of unit test pass rates.

3.2 Model Selection

In this study, we evaluate and compare several state-of-the-art LLMs: StarCoder2-15B [36, 41], GPT-4o and GPT-4o Mini [52], LLaMA 3 8B [76], and DeepSeek-v3 [19]. StarCoder2-15B is chosen because its training dataset is open-source, allowing us to account for data leakage in our experiments. StarCoder2 is also selected due to its training on a diverse corpus of programming tasks, making it particularly adept at generating syntactically correct and contextually relevant code suggestions [36]. Additionally, GPT-4o and GPT-4o Mini are included because they represent widely used, high-performing general-purpose LLMs commonly applied in real-world software development settings [52]. We further include LLaMA 3-8B, known for its efficiency and effectiveness in code generation despite its smaller parameter count, offering insights into the performance trade-offs between model size and code quality [76]. Finally, we test DeepSeek-v3 as it is a recent open-source model that demonstrates strong reasoning performance and competitive code-generation ability. Overall, we examine a diverse set of LLMs: open-source and closed-source, code-specialized and general-purpose, and small to medium parameter sizes, allowing a comprehensive comparison of their refactoring effectiveness in realistic software engineering tasks.

3.3 Data Preparation

We conduct our study on a state-of-the-art refactoring dataset [48] comprising 196 Java projects, constructed from the 20-MAD dataset [16]. The dataset includes the development and refactoring information extracted by the Rminer [78] tool, such as refactoring types. The accuracy of the refactorings detected and included in this dataset is reported with a precision of 99.7% and a recall of 94.2%, making it suitable for our study.

Similar to the previous work [48], we limit our study to projects primarily written in Java. Selecting projects using Java is based on the strong performance of refactoring detection [77, 78] and code smell extraction [67] tools developed for Java. Furthermore, Java is one of the popular programming languages [6, 29] among developers [48], which aligns with our goal of examining refactorings in a language where refactoring practices are well-established and widely adopted.

To account for data leakage, we filter out the 136 Java projects from our initial dataset that are included in the training dataset of StarCoder2. Moreover, to ensure a balanced distribution of refactoring commits across projects, we filter out projects with fewer than the median number (*i.e.*, 129) of refactoring commits, resulting in a final 30 Java projects for our analysis. This treatment is primarily adopted to ensure a sufficient number of commits per project and to exclude projects with minimal refactoring activity. The summary statistics for all 30 projects are presented in Table 1.

To ensure that we extract pure refactoring commits, rather than other tasks such as development or bug fixing, we select commits where 100% of the lines of code churn consist of refactoring operations. Refactoring code churn refers to the sum of lines added and removed in the process of a refactoring operation. To measure refactoring churn, we select code segments in terms of lines of code that each refactoring operation starts and ends on. We then calculate the ratio of refactoring code churn to total code churn in each commit, including refactorings within or related to the new classes or files, to identify commits that exclusively consist of refactoring operations. For instance, extract class refactoring may introduce new methods and classes, the code churn associated with these transformations is still captured under refactoring operations. If the code churn is entirely due to refactoring (*i.e.*, Refactoring Ratio = 100%), then the commit is classified as a refactoring commit. Refactoring Ratio of Commit is computed using the following equation [48]:

$$\text{Refactoring Ratio of Commit } (i) = \frac{\text{Refactoring Code Churn of Commit } (i)}{\text{Total Code Churn of Commit } (i)} \times 100\% \quad (1)$$

From each of our selected projects and their refactoring commits, we randomly select up to 334 commits using a 95% confidence level and 5% margin of error, resulting in a total of 5,194 refactoring commits along with

Table 1. Summary of Selected Projects

Repository Name	Forks Count	Stars Count	Age (Years)	Total Lines of Codes	Commits Count	Refactoring Commits Count
camel	4,920	5,440	15	2,630,382	71,671	760
activemq	1,443	2,285	15	597,401	11,821	546
accumulo	445	1,057	13	486,047	12,599	483
james-project	464	860	9	595,437	15,629	798
openmeetings	254	630	9	98,840	3,716	181
jmeter	2,066	8,174	14	217,309	18,266	554
skywalking	6,479	23,593	9	160,728	8,041	166
jclouds	135	193	10	563,320	10,878	699
qpid-jms-amqp-0-x	10	11	7	116,911	7,631	374
attic-polygene-java	34	91	9	237,036	5,820	303
incubator-brooklyn	93	140	10	329,178	13,002	541
incubator-druid	3,668	13,355	12	1,447,251	14,281	538
systemml	463	1,029	9	529,475	8,603	383
oozie	472	704	13	247,554	2,409	151
apex-malhar	152	132	9	268,526	6,097	379
ode	61	44	15	130,971	3,823	136
incubator-gobblin	745	2,208	9	391,556	6,409	365
servicecomb-pack	435	1,926	7	37,016	1,683	139
falcon	117	100	11	153,672	2,227	141
myfaces-extcdi	12	9	14	58,274	1,136	130
incubator-pinot	1,238	5,314	10	657,844	11,950	498
brooklyn-library	37	11	9	56,662	3,811	134
deltaspike	137	149	11	122,070	2,671	192
incubator-iotdb	982	4,353	6	936,994	11,020	212
apex-core	176	351	9	94,190	6,122	407
myfaces-trinidad	10	4	7	345,041	4,730	172
incubator-shardingsphere	6,675	19,692	8	460,393	43,295	2,527
incubator-dolphinscheduler	4,533	12,548	5	223,215	8,473	161
incubator-taverna-language	26	22	9	51,881	3,560	158
hadoop-ozone	487	813	5	658,977	7,916	264

23,199 changed files. Among these commits, 57.84% involve changes to more than one file, confirming that a substantial portion of real-world refactorings require coordinated, cross-file transformations. Random sampling is a practical and widely accepted strategy in large-scale LLM evaluations [48, 65, 83] to ensure representativeness while keeping the analysis computationally feasible. For instance, the StarCoder2 LLM takes, on average, about 8 minutes to generate a refactoring for each commit. Our final dataset reaches a reasonable size, comparable to or exceeding that of previous studies on LLMs [20, 58, 69].

3.4 Quality Assessment

In this section, we provide the measures used to assess the quality of the generated refactorings, including unit tests, code smells, and static code metrics.

3.4.1 Code Smells. Code smells are indicators of underlying design or implementation issues that may affect the quality, such as the maintainability and readability of the software [48]. Code refactoring is commonly associated with the elimination of code smells [7, 10, 70, 77, 87]. We use DesigniteJava 2.5.2 [67] to extract code smells by inputting the source code of each Java file from before and after each refactoring commit. DesigniteJava offers comprehensive coverage in detecting code smells across multiple categories [66]. It can detect 46 different types of code smells, including 7 architecture smells, 18 design smells, 9 implementation smells, 4 testability smells, and 8 test smells.

- Architecture smells describe the characteristics in the system's architecture that indicate potential issues affecting the overall software quality. For example, unstable dependencies code smell describes the case that relies on frequently changing modules.
- Design smells indicate poor adherence to design principles that can negatively impact the software's modularity, flexibility, and reusability. These smells suggest that the design may not support future changes or scale effectively, leading to increased technical debts and higher maintenance costs. For example, unnecessary abstraction describes the code smell with too many layers or indirections that could obscure the code's intent.
- Implementation smells are signs in the source code to flag problematic code that could make the code harder to maintain, understand, or extend. Common examples include large classes, repeated code, or very long methods, which indicate that the code may need improvement to enhance its quality and ease of maintenance.
- Testability smells refer to the degree to which the development of test cases can be facilitated by the software design choices. Examples include hard-wired dependencies and excessive dependencies.
- Test smells are a result of bad programming practices in unit test code, indicating potential design problems in the test code. Examples include empty test, unknown test, and constructor initialization.

Code smells are typically detected using predefined thresholds and rules in code [67]. For example, a method containing more than 100 statements is considered a *Long Method. Long Method* code smell can be addressed by any refactoring that reduces the number of statements, such as *Split Method, Split Conditional*, or *Extract Class*. Therefore, regardless of the specific refactoring type applied, there is no strict one-to-one mapping or fixed sequence between refactoring types and code smells [7]. The elimination of code smells generally reflects healthier code and serves as an indicator of successful refactoring [1, 17, 24]. We use these extracted smells to compute the Code Smell Reduction Rate (SRR), a metric that quantifies how effectively refactoring reduces the total number of detected code smells.

3.4.2 Code Metrics Computation. While code smells are more closely associated with refactoring, code metrics complement code smells [49] and provide more general insights into the overall quality of the code, such as maintainability, and complexity [50]. We collect code metrics that are used to evaluate complexity, cohesion, coupling, and modularity [49]. The description of the collected code metrics is listed in Table 2. Modularity metrics help assess the degree to which the system is divided into independent components, which promotes ease of maintenance and testing. Coupling metrics indicate the degree of interdependence between classes, where lower coupling is preferred for better modularity and flexibility. Cohesion metrics measure how closely related the responsibilities of a class are, with higher cohesion typically leading to improved code clarity and design quality [55]. We use the Understand tool [62], which is a static code analysis tool to extract code metrics.

We derive combined metrics to represent each quality attribute separately: for coupling, we average *Count Class Coupled* and *Count Class Coupled Modified*; for modularity, we average *Count Class Derived*, *Count Decl Class Variable*, and *Count Decl Instance Variable*; for cohesion, we average *Percent Lack of Cohesion* and *Percent Lack of Cohesion Modified*; and for complexity, we use only cyclomatic complexity (*i.e., Cyclomatic* in Table 2).

Table 2. Description of Metrics Used in the Study [63].

Metric	Description	Quality Attribute	Rationale for Inclusion
Count Class Coupled	Number of other classes coupled to.	Coupling	Helps identify tightly coupled classes, indicating potential areas for refactoring to reduce complexity.
Count Class Coupled Modified	Number of other non-standard classes coupled to.	Coupling	Focuses on the complexity introduced or modified during changes, relevant for assessing refactoring impact.
Count Class Derived	Number of immediate subclasses.	Modularity	High values may indicate a deep inheritance hierarchy, affecting maintainability and ease of modification.
Count Decl Class Variable	The number of class-level variables declared in a class.	Modularity	A large number of class variables can reduce cohesion and increase the potential for errors.
Count Decl Instance Variable	The number of instance variables declared in a class.	Modularity	High instance variable counts may signal a class doing too much, reducing its maintainability.
Percent Lack of Cohesion	The percentage of methods in a class that do not share instance variables.	Cohesion	Indicates the degree of cohesion in a class, with higher values suggesting poor design and low cohesion.
Percent Lack of Cohesion Modified	A variation of PercentLackOfCohesion, modified for accessor methods.	Cohesion	Assesses the impact of changes on class cohesion, useful for understanding the effects of refactoring.
Avg Cyclomatic	The average cyclomatic complexity for all nested functions or methods.	Complexity	Provides an overall view of the code's complexity, helping to identify areas that may need simplification.
Cyclomatic	The McCabe cyclomatic complexity of a single method or function.	Complexity	Directly measures the complexity of a method, indicating potential difficulties in testing and understanding.
Max Cyclomatic	The maximum cyclomatic complexity of any method in a class or project.	Complexity	Highlights the most complex methods, which could be refactoring targets to improve maintainability.
Sum Cyclomatic	The total cyclomatic complexity of all methods in a class or project.	Complexity	Summarizes the overall complexity, aiding in assessing the overall maintainability and testability of the codebase.

3.4.3 Tests Preparation. Code refactoring should not alter the external functionality of the code [24]. To evaluate whether the refactorings generated by LLMs retain their original functionality and to ensure that semantic correctness is validated, we use and execute unit tests on the refactored code. The unit tests serve as a check for behavioral equivalence and help filter out hallucinated or semantically incorrect transformations that may compile but change program behavior. In our dataset, 9,687 of the target files include the original test cases, however, the remaining 13,512 do not include the original test cases. Therefore, we generate test cases for the remaining 13,512 target files in refactoring commits that are missing test cases. To this end, we utilize EvoSuite [25], which performs static analysis to generate test suites that are optimized to maximize code coverage across multiple criteria, including line coverage (*i.e.*, ensuring all lines of code are executed), branch coverage (*i.e.*, testing each possible path in the code), and output coverage (*i.e.*, verifying the correctness of program outputs). EvoSuite incorporates JUnit 4 assertions, which are used to capture and validate the current behavior of the tested classes. We take the following steps to generate test cases using EvoSuite:

(1) **Clone the repository:** For each refactoring commit, we check out the repository at the state immediately before the refactoring is applied. To ensure the project is correctly configured, we extract the top-level `pom.xml` file from the repository to rebuild the original dependency graph. The pom.xml is an XML configuration file used by Apache Maven [45] to manage dependencies, build settings, and other project configurations. Proper configuration is essential to compile and execute the project successfully. When the root `pom.xml` does not declare required modules, we recurse through submodules to locate the correct configuration. This ensures that generated tests execute within the same dependency environment, library versions, and build settings as the original project.
(2) **Compile the project:** Once the project is set up, we compile the entire project source code using Apache Maven to verify that it builds successfully. This step ensures that all required dependencies are correctly

resolved and that the code is free of compilation errors. A successful compilation is a prerequisite for generating valid test cases.
Once the project is set up, we compile the entire project using Apache Maven, ensuring that all modules, dependencies, and external libraries are resolved. Test generation is performed only after a successful build, guaranteeing that EvoSuite has access to the correct type hierarchy, external APIs, and runtime scaffolding. Importantly, we do not mock dependencies during this step as EvoSuite interacts with the actual classes, including external library calls, unless the project itself uses mocks.

(3) **Generate test cases with EvoSuite:** EvoSuite generates tests directly against the compiled classes, including invocation of external library methods where applicable. This minimizes artificial isolation so that EvoSuite observes the real object interactions available in the original codebase and encodes these behaviors into JUnit assertions that reflect the project's true runtime semantics.

Furthermore, we manually validate the generated unit tests to test their correctness. Thus, we randomly select 374 unit tests from 23,199 (*i.e.,* 95% confidence and 5% margin of error) of the generated unit tests. The first and second authors then perform open coding and evaluate the test cases to determine their validity and functionality. Of the 374 unit tests, 352 tests (94.1%) are found to be valid and functional. The remaining test cases (5.9%) are identified as invalid due to the following issues:

- Overly Strict Assertions (65% of observed failures): EvoSuite generates assertions that are too rigid, for example, expecting a list in a specific order even when the method only guarantees that certain elements are present. This leads to false negatives when harmless variations occur.
- Non-deterministic (Flaky) Tests (25% of observed failures): Some tests show non-deterministic behavior, where the same test may pass in one run and fail in another. This is due to the test relying on aspects of the system state, such as timestamps or random number generation (from the 5 observed cases of failure). For example, one test compares a dynamically generated timestamp to a fixed expected value, which causes intermittent failures.
- Incomplete Test Generation (10% of observed failures): Due to the search budget (*i.e.,* time limit for test generation) of 60 seconds, some unit tests are left incomplete. In one example, we see a test that calls a method of an object that has not been properly instantiated, leading to a runtime exception (*i.e.,* NullPointerException).

After we perform manual validation, we filter out unit tests that match patterns of failing tests. We distill each failure category into one or more regular expression filters. We compile these patterns using Java's Pattern and Matcher classes and automatically exclude any test whose failure message matches any filter.

3.5 Practical Deployment Cost

All experiments were executed on a server equipped with 4 × Nvidia A100 GPUs (80 GB each) or used the API available at the authors' research laboratory. Generating refactorings for the full dataset of 5,194 commits using StarCoder2 required approximately 7 days of continuous execution, corresponding to an average throughput of roughly 8 minutes per commit. This provides a realistic lower bound on processing time for large-scale multi-file refactoring pipelines under similar hardware conditions.

The computational cost of refactoring is influenced by model size and architectural complexity. Smaller open-source models (e.g., StarCoder2, LLaMA-3-8B) incur lower memory and per-token inference overhead, but exhibit reduced refactoring reliability and lower test-pass rates. In contrast, larger production-grade models (e.g., GPT-4o, DeepSeek-v3) demonstrate substantially higher functional correctness, but at increased inference cost and latency. In addition, prompt-engineering strategies directly affect computation. One-shot RAG and chain-of-thought prompting expand prompt length and sampling budget relative to zero-shot prompting, increasing total token usage and inference time per refactoring attempt.

Before Refactoring

```
public class SimpleCamelServletContextListener
↪  extends CamelServletContextListener {

    private Map map;

    @Override
    public Registry createRegistry() throws
    ↪  Exception {
        map = new SimpleRegistry();
        return (Registry) map;
    }

    /**
     * Gets the {@link Map} that contains the
     ↪  data for the {@link SimpleRegistry}
     */
    public Map getMap() {
        return map;
    }
}
```

After Refactoring

```
public class SimpleCamelServletContextListener
        extends
        ↪  CamelServletContextListener<SimpleRegistry>
        ↪  {

    private SimpleRegistry map;

    @Override
    public SimpleRegistry createRegistry() throws
    ↪  Exception {
        map = new SimpleRegistry();
        return map;
    }

    /**
     * Gets the {@link Map} that contains the data
     ↪  for the {@link SimpleRegistry}
     */
    public Map<String, Object> getMap() {
        return map;
    }
}
```

Fig. 3. Example of a refactoring in the `camel` project performed by a developer. The refactoring updates the types of attributes and methods from generic `Map` and `Registry` to their concrete types `SimpleRegistry`. This improves type safety, meaning potential type mismatches are caught at compile time rather than causing runtime errors. For example, using `Map<String, Object>` instead of a raw `Map` ensures that keys and values conform to expected types, reducing the chance of `ClassCastException`.

4 Results

In this section, we provide the motivation, approach, and findings for each of our research questions.

4.1 RQ1: What is the performance of LLMs in code refactoring?

4.1.1 ***Motivation.*** Developers typically perform refactoring by manually analyzing and editing code to improve maintainability and readability [23, 24]. This requires developers to rely on their expertise and knowledge of best practices, which can make the process both time-consuming and prone to errors [44]. In contrast, LLMs generate refactorings by processing the prompt and code through pretrained models that are trained on vast amounts of data to automatically suggest refactorings. In this research question, we aim to assess the performance of LLMs in refactoring tasks by comparing the quality of the refactorings they produce.

4.1.2 ***Approach.*** Figure 3 shows an example of a developer refactoring. As discussed in Section 3.3, we select a total of 5,194 refactoring commits containing a total of 23,199 files to test and compare different LLM-generated refactorings. We then generate refactorings using our selected LLMs, namely, StarCoder2, GPT-4o and GPT-4o Mini, LLaMA 3, and DeepSeek-v3. Finally, we use code smells, code metrics, and unit tests to compare the quality of the code refactored by the LLMs.

Figure 4 shows an example of a refactoring generated by StarCoder2 LLM in the `camel` project. The changes include updates to variable declarations, method return types, and the use of generics to improve type safety (*i.e.,*

Before Refactoring

```
public class SimpleCamelServletContextListener
 ↪  extends CamelServletContextListener {

    private Map map;

    @Override
    public Registry createRegistry() throws
     ↪  Exception {
        map = new SimpleRegistry();
        return (Registry) map;
    }

    /**
     * Gets the {@link Map} that contains the
      ↪  data for the {@link SimpleRegistry}
     */
    public Map getMap() {
        return map;
    }
}
```

After Refactoring

```
public class SimpleCamelServletContextListener
 ↪  extends CamelServletContextListener {

    private final Map<String, Object>
     ↪  registryMap = new SimpleRegistry();

    @Override
    public Registry createRegistry() throws
     ↪  Exception {
        return (Registry) registryMap;
    }

    /**
     * Gets the {@link Map} that contains the
      ↪  data for the {@link SimpleRegistry}
     */
    public Map<String, Object>
     ↪  getRegistryMap() {
        return registryMap;
    }
}
```

Fig. 4. Example refactoring in the camel project performed by StarCoder2. The refactoring improves type safety by using a parameterized `Map<String, Object>` instead of a raw `Map`; therefore, the compiler can catch type mismatches at compile time. The refactoring enhances immutability by marking the map as `final`, ensuring the reference cannot be reassigned. The variable is also renamed to improve semantic clarity.

reducing the risk of runtime ClassCastExceptions by ensuring compile-time type checks). In this example, the untyped Map map is replaced with a strongly typed Map<String, Object>, clarifying the expected key-value types. This use of generics prevents potential misuse of the map (*e.g.,* inserting unintended types) but also improves readability and tooling support (*e.g.,* autocompletion and static analysis). Additionally, renaming the method getMap() to getRegistryMap() and marking the map as final improves maintainability by enforcing immutability and better expressing the variable's role.

Refactoring Generation: We provide the LLMs with the original, unrefactored code as input and compare their generated refactored output against the refactoring applied by developers. The prompt includes multiple code snippets from different files within the same commit, allowing the model to perform multi-file refactoring rather than being limited to single-file scenarios. The prompt does not include the test case to ensure the LLM does not overfit to the test case. This setup ensures that the LLMs receive sufficient context to refactor code spanning across multiple files, mirroring real-world refactoring tasks.

Figure 5 shows the zero-shot prompt template, which is a method where the model receives no additional examples or guidance beyond the initial instruction [33], used to instruct LLMs to take the developers' code snippet before refactoring as input. We employ the "###" symbol as a delimiter to ensure the LLM focuses on the instructions even with a block of code in the prompt [90]. The LLM is specifically prompted to refactor Java code, but is not instructed to remove code smells or improve specific code metrics, ensuring its focus remains on refactoring rather than overfitting to code smell elimination. We create a zero-shot prompt for each refactoring commit using a Python script, which is loaded onto an Nvidia A100 GPU.

Instruction
You are an advanced AI model specialized in refactoring Java code. Code refactoring is the process of improving the structure, readability, and maintainability of a codebase while preserving its original functionality. Given a set of related Java code snippets from multiple files before refactoring, you must output the improved, refactored version of each snippet.

Prompt
The following Java code snippets are from different files within the same commit and are part of a refactoring operation:

File: {file_name_1}
{code_segment_before_refactoring_1}

File: {file_name_2}
{code_segment_before_refactoring_2}

Fig. 5. Prompt Used to Instruct LLMs to Perform Refactoring

Semantic Validation of Refactored Code: To ensure that LLM-driven refactorings preserve the original behavior, we apply a two-step validation. First, we randomly sample 68 commits (*i.e., 90% confidence with 10% margin of error* containing developer test cases. We then generate EvoSuite unit tests for each of these commits and compare pass/fail outcomes.

Test Pass Rate Evaluation: We use Pass@1, Pass@3, and Pass@5, all using the same zero-shot prompt, to assess the functional correctness of the refactoring operations. These metrics reflect how likely an LLM is to produce a functionally correct refactoring within one, a few, or several attempt:

- **pass@1:** We generate a single refactored solution for each code snippet in a commit. This solution is evaluated by executing the corresponding unit tests. If the refactored solution passes the unit test, it is considered a successful refactoring under the Pass@1 criterion.
- **pass@3:** We generate three refactored solutions for each code snippet in a commit. Each solution is independently evaluated by running the corresponding unit tests. If at least one of these three refactored versions passes all unit tests, the refactoring is considered successful under pass@3.
- **pass@5:** We generate five refactored solutions for each code snippet in a commit. As with pass@3, we run the unit tests on each of the five refactored versions. If at least one solution successfully passes all unit tests, the refactoring is deemed successful under pass@5.

In pass@3 and pass@5 scenarios, if multiple refactoring solutions pass the unit test, we select the best solution by analyzing which solution reduces the most amount of code smells. If two or more solutions pass the unit tests and reduce the same amount of code smells, we select the best solution based on which one has the highest improvement in code metrics (*i.e.* cohesion, coupling, modularity, and complexity).

Quality assessment: To evaluate whether LLM-generated refactorings improve software quality by removing design issues, we measure the Code Smell Reduction Rate (SRR) across three versions of the code: (1) the unrefactored code, to assess its quality before refactoring; (2) the developer-refactored code, to evaluate its quality after developer refactoring; (3) and the LLM-refactored code, to assess its quality after LLM refactoring. We detect code smells using DesigniteJava [67], which is a static analysis tool that identifies a wide range of object-oriented

design smells discussed in Section 3.4.1. This analysis allows us to systematically compare the effectiveness of refactorings carried out by the LLMs. For calculating code smell reduction rate (SRR), let A_{before} represent the total number of code smells before refactoring in all files changed by the commit, and A_{after} represent the total number of code smells after refactoring in all files changed by the commit. We calculate the code smell reduction rate using the following formula:

$$SRR = \frac{A_{\text{before}} - A_{\text{after}}}{A_{\text{before}}} \times 100 \tag{2}$$

Then, using the Scott-Knott Effect Size Difference (ESD) [73, 74] test on the SRR distributions across 30 projects, we examine the statistical difference in the quality of refactorings produced by the LLM and developers. The Scott-Knott ESD test is a non-parametric hierarchical clustering method that partitions multiple distributions into statistically distinct groups. The Scott-Knott ESD test is particularly suited for comparing multiple techniques, as it ranks them based on effect size while controlling for statistically significant differences. For each of the 30 projects, we compute the code smell reduction rates for each refactoring method (e.g., each LLM and developers). The Scott-Knott ESD test then partitions these distributions into ranked clusters, with each cluster representing methods that are not significantly different from one another. In this ranking system, a lower rank (*i.e.,* 1) indicates a superior performance. To this end, we test the following null hypothesis:

- ***H_0:*** *The refactorings performed by the GPT-4o and GPT-4o Mini, LLaMA 3, StarCoder2, and DeepSeek-v3 do not have significant differences in reducing code smells.*

Addressing Data Leakage: To examine whether LLM performance might be inflated due to data leakage, we conduct an analysis on the subset of projects known to appear in the StarCoder2 training dataset. StarCoder2 is the only model in our study with a publicly documented training corpus, allowing us to identify projects contained in the training set of StarCoder2. We execute the RQ1 evaluation pipeline (unit tests and SRR) on this subset and compare the results to the unseen projects. Because the other LLMs in our study are closed-source and do not disclose full training sources, we report their performance on the same five projects for reference but do not treat these as definitive indicators of training overlap.

Developer Analysis: To better understand how refactoring practices vary across contributor roles, we categorize developers based on their level of engagement within each project. We categorize developers in a project using the following criteria based on the "onion model" [47]:

- Core developers: Developers who belong to the top quantile (default top 10%) in terms of commit counts. These contributors are responsible for the majority of commits and typically have the most influence on the project's development and maintenance.
- Peripheral developers: Developers who have contributed multiple commits but do not belong to the top quantile of commit activity. These developers participate in the project but at a lower frequency compared to core developers.
- One-time contributors: Developers who have made only a single commit to the project. These contributors are often occasional participants or external collaborators who make minor contributions without ongoing involvement.

Intention to Refactor: To ensure a fair comparison and better understand the motivations behind refactorings performed by developers, we analyze the intention behind each commit. To this end, we first use an LLM to generate a natural language summary of the likely refactoring intention based on the commit message and code changes [54, 75]. These generated intentions are then embedded using the SentenceTransformer model (all-MiniLM-L6-v2) [80] and clustered using KMeans [28] to identify if similar intents from different refactoring operations can be categorized within the optimal number of meaningful clusters. We determine the optimal number of clusters by computing silhouette scores [64] across a range of clusters from 1 to 20 and find the value with the highest score as the optimal number of clusters. The optimal number of clusters with the highest

Table 3. Comparison of Unit Test Pass Rates (Median) and SRR Across the 30 Projects.

Refactoring	Approach	Unit Test Pass Rate (Median)			SRR	
		pass@1	pass@3	pass@5	Median	Average
LLM	StarCoder2	33.0%	43.1%	49.6%	4.93%	6.48%
	GPT-4o Mini	78.5%	84.3%	88.0%	6.78%	7.89%
	LLaMA 3	73.2%	78.0%	82.5%	**15.15%**	**15.73%**
	DeepSeek-v3	82.1%	87.0%	91.8%	10.42%	11.05%
	GPT-4o	**85.4%**	**90.2%**	**93.7%**	12.21%	12.84%
Developer	-	100%	-	-	1.65%	2.28%

silhouette score (0.535) [48, 60] is found as three, representing the three main intents behind refactoring. The first author manually interpreted and labeled these clusters by reviewing the GPT-generated reasons associated with the refactoring commits in each cluster and identifying common themes. Based on this review, the clusters are labeled as: (1) Code Organization & Standardization, (2) Code Quality & Maintainability Enhancements, and (3) Test Stability. We then analyze the quality of refactoring within each developer intention category.

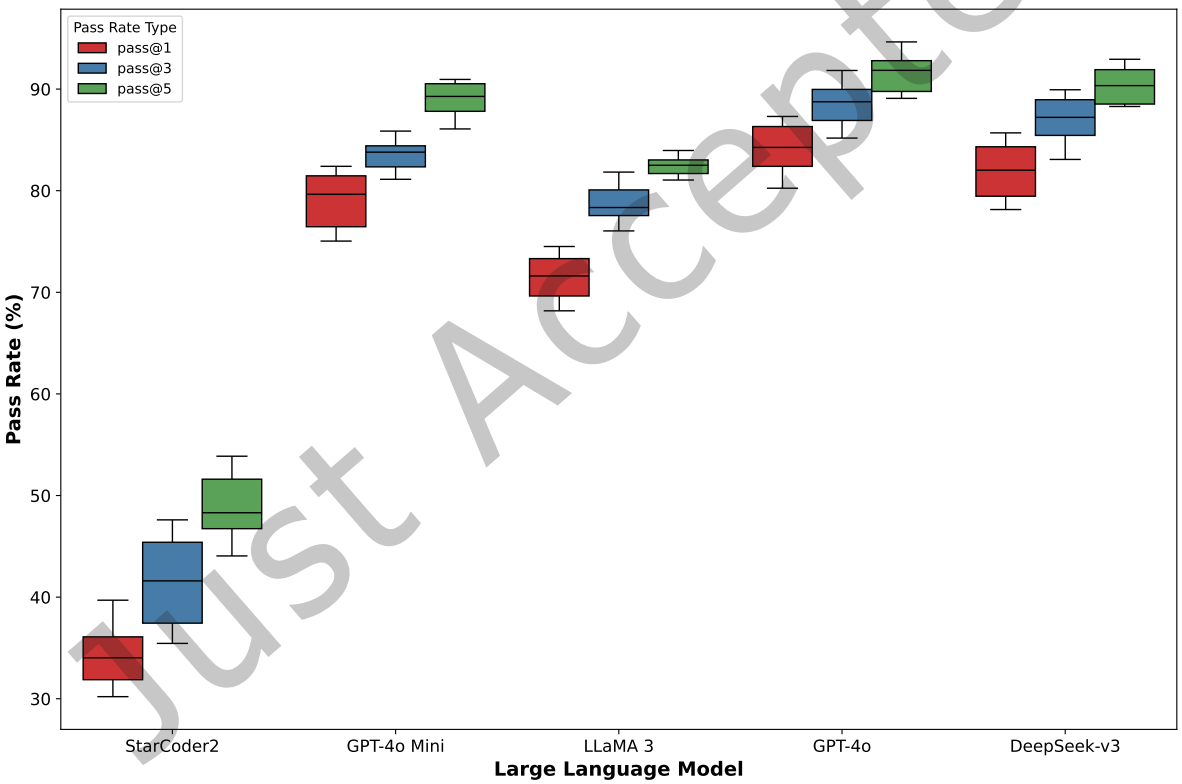

Fig. 6. Distribution of Unit Test Pass Rates Across 30 Software Projects After LLM-Generated Refactorings.

4.1.3 ***Findings.*** **Refactorings generated by LLMs can hallucinate changes that affect the functionality of the code in 39% of the cases.** To provide a detailed comparison of the refactoring performance between LLMs, as shown in Table 3, we present the unit test pass rates alongside the code smell reduction rate (SRR) for our

Table 4. Performance Comparison on the 5 Projects Included in the StarCoder2 Training Dataset ("Seen Projects").

Refactoring	Approach	Unit Test Pass Rate			SRR	
		pass@1	pass@3	pass@5	Median	Average
LLM	StarCoder2	35.4%	45.2%	51.8%	5.12%	7.21%
	LLaMA 3	71.9%	77.4%	81.3%	14.87%	15.08%
	GPT-4o Mini	80.1%	85.0%	88.9%	6.95%	8.14%
	DeepSeek-v3	83.0%	87.8%	92.1%	9.65%	10.42%
	GPT-4o	86.2%	91.1%	94.4%	11.34%	12.02%

LLMs. LLMs exhibit varying performance across different models. Figure 6 visualizes the distribution of unit test pass rates across projects for each model. Among the models evaluated, GPT-4 achieves the highest median unit test pass rate (*i.e.,* pass@1) at 78.5%, followed by LLaMA 3 at 73.2%. StarCoder2 demonstrates lower success rates, which may be attributed to the complexity of multi-file refactorings and the absence of training data containing similar code, unlike production-grade models that may have been trained on the projects in our evaluation (*i.e.,* data leakage). The higher unit test pass rates of LLaMA 3 and GPT-4 reflect improved performance over StarCoder2, demonstrating the advantages of production-grade models (*i.e.,* LLMs that are deployed to operate effectively in a real-world environment) with enhanced reasoning [26] and code generation capabilities. In our extended evaluation that includes DeepSeek-v3 and GPT-4o, we observe even stronger performance. GPT-4o achieves the overall highest pass@1 rate at 85.4%, with DeepSeek-v3 following at 82.1%, both surpassing GPT-4o Mini and LLaMA 3. These results indicate that next-generation models continue to push refactoring reliability, especially in multi-file transformations where structured reasoning is critical.

Table 4 compares LLM performance on the five randomly selected projects included in the StarCoder2 training dataset. StarCoder2 shows only minor improvements on seen projects relative to unseen ones (pass@1: 35.4% vs 33.0%; SRR: 7.21% average vs 6.48% average). This suggests that, for StarCoder2, potential training overlap (*i.e.,* data leakage) does not appear to have a large impact on refactoring performance and that the model is not simply memorizing refactoring solutions. The small differences indicate that refactoring in this setting is not primarily driven by recall, but instead requires reasoning over the current code context, which often differs from any training snapshot. However, this analysis is limited to StarCoder2, whose training exclusions are known, and does not constitute proof that larger, closed-source models are unaffected by memorization or data leakage effects.

Our analysis of 68 EvoSuite-generated unit test suites to assess whether LLM-driven refactorings preserve semantic behavior indicates high reliability in EvoSuite tests. From the evaluation outlined in Section 4.1.2, of the 68 commits with developer-provided test suites (210 total assertions), EvoSuite's generated tests agreed with the developer tests 94.3% of the time, confirming that the refactored code maintains the original functionality.

We also evaluate pass@3 and pass@5, which represent the percentage of refactorings where at least one of the top-3 or top-5 completions passes all unit tests. These metrics show that pass rates increase with the number of attempts, suggesting that even when the first suggestion fails, viable alternatives often exist among the top completions. Table 3 shows the pass@3 and pass@5 rates for the LLMs. StarCoder2 improves from 33.0% at pass@1 to 43.1% at pass@3 and 49.6% at pass@5; LLaMA 3 rises from 73.2% for pass@1 to 82.5% for pass@5; and GPT-4 climbs from 78.5% for pass@1 to 88.0% for pass@5. Our extended results show that DeepSeek-v3 increases from 82.1% (pass@1) to 91.8% (pass@5), and GPT-4o achieves the highest overall improvement, rising from 85.4% (pass@1) to 93.7% (pass@5). These gains indicate that newer models not only perform better on first attempts but also provide increasingly strong alternative refactorings among their top completions. Across all models, pass@5 consistently yields the highest unit test pass rates. **Therefore, it is highly recommended to integrate comprehensive testing before utilizing LLMs for automated refactoring, as their use without proper testing may alter or break the program's logic.** To this end, we use the results of pass@5 for the remainder of the study, as it represents the best refactoring outcome among multiple attempts, allowing us to evaluate the

maximum potential of each LLM in terms of code smell reduction and code quality improvement. This ensures consistency across all analyses while focusing on the most effective refactorings produced by each LLM.

To identify the causes of test case failures, we randomly sample and examine 100 refactorings performed by LLaMA 3, GPT-4o Mini, and StarCoder2. The main causes of these failures are listed in Table 5. Approximately 30% of the failures stem from unintended functionality changes, while errors related to variable management (addition, renaming, and scoping) account for nearly 53% of all failures. The remaining cases are due to data type handling issues, incomplete refactorings, and syntax errors.

Table 5. Main causes of test case failures after LLM-based refactoring

Cause of Failure	**Failure Rate**
Introduction of unintended functionality	30%
Addition of unnecessary or irrelevant variables	20%
Incorrect or inconsistent renaming of variables or methods	18%
Improper variable scoping or naming conflicts	15%
Incorrect handling or casting of data types	8%
Incomplete refactoring (partially modified code)	5%
Syntax or structural errors	4%

Understanding failure patterns suggests several ways to improve LLM-generated refactorings. Incorporating static code analysis during or after the refactoring process can help detect unintended functionality changes, variable naming conflicts, and other structural issues before the code is merged. For example, static analysis tools such as SpotBugs, PMD, or Error Prone can verify that all control structures (e.g., if, for, while) are properly closed, that variable types are consistent, or that no references are made to undefined or renamed variables. Another way LLM refactoring can be improved is by adopting iterative prompting strategies, where the LLM's intermediate outputs are validated against expected outcomes, which could reduce errors related to incomplete refactoring and data type mismanagement. Furthermore, fine-tuning LLMs with high-quality, multi-file refactoring examples may enhance their understanding of complex code structures, thereby mitigating failures associated with variable management and unintended code modifications. Another effective strategy to improve LLM refactorings is to conduct an analysis of each error category, enabling fixes such as automated patch generation for inconsistent renaming and for resolving variable visibility conflicts that cause out-of-scope references.

To explore whether LLMs can self-correct after failing unit tests, we perform a preliminary investigation into the effectiveness of providing error logs as direct feedback to StarCoder2. In this exploratory experiment, when a refactored piece of code generated by the LLM fails an associated unit test, we capture the resulting error logs as textual feedback. This feedback is provided back to StarCoder2, explicitly instructing it to address and correct the identified issues. StarCoder2 is prompted to reconsider its refactoring decisions based on the error details and attempt a corrected refactoring. Through this feedback loop, we observe that out of the total 660 attempted corrections prompted by error log feedback, 50 previously failing cases successfully passed the unit tests. Although the overall success rate indicates there remains room for improvement, these initial results demonstrate that providing feedback in the form of error logs can help LLMs better identify their mistakes, guide their corrective reasoning, and lead them toward more reliable refactoring outcomes. Future work could explore automated mechanisms for capturing, structuring, and feeding back detailed diagnostic information to LLMs, potentially integrating with continuous integration (CI/CD) pipelines or integrated development environments (IDEs) to support iterative refactoring.

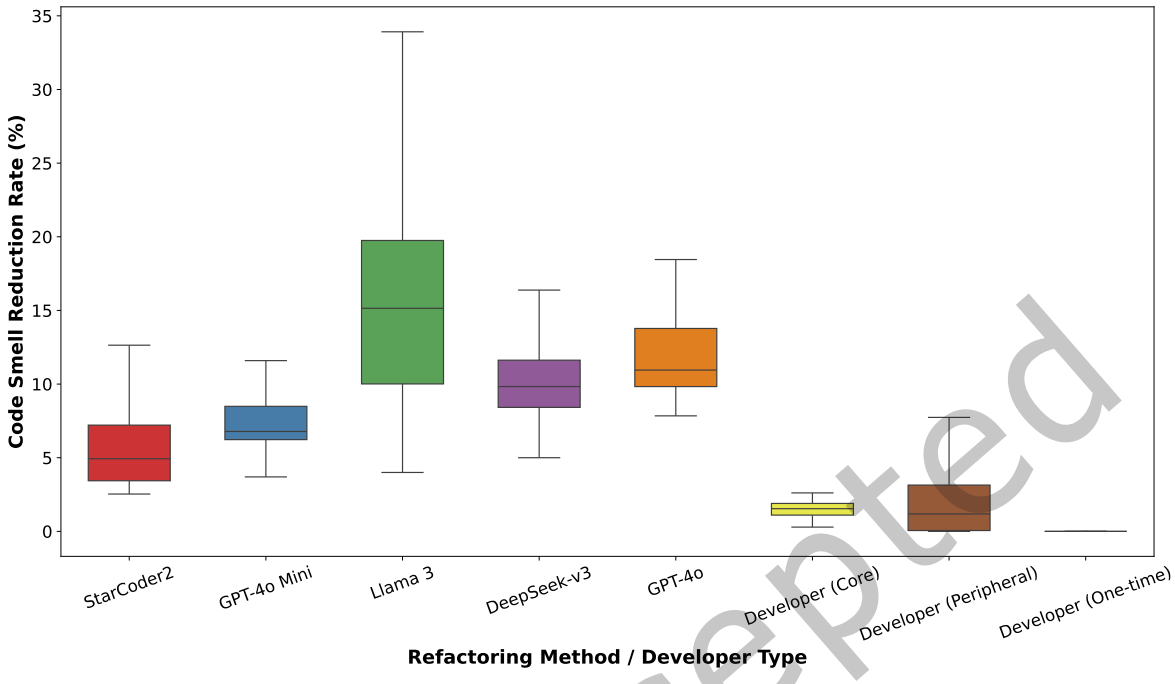

Fig. 7. Distribution of Code Smell Reduction Rates Across 30 Projects for LLM-Generated Refactorings and Developer Refactorings.

LLMs improve code quality as much as 15% through refactoring when the generated code passes unit tests. Figure 7 shows the distribution of code smell reduction rates (SRR) for developers, StarCoder2, LLaMA 3, and GPT-4. The Scott-Knott-ESD test results show a significant difference in the distribution of SRR across refactorings performed by LLaMA 3, GPT-4, StarCoder2, and developers. LLaMA 3, classified as Cluster 1, achieves the highest ranking in the Scott-Knott ESD analysis, with a median SRR of 15.15%, indicating it is the most effective in reducing code smells. This result suggests that production-grade models with strong reasoning capabilities may outperform other LLMs in improving code quality. GPT-4o achieves a median SRR of 12.21%, and DeepSeek-v3 achieves 10.42%, positioning both models between LLaMA 3 and GPT-4o Mini at cluster 2. GPT-4o Mini and StarCoder2, grouped in Cluster 3, also show improvements over developers, with median SRRs of 6.78% and 4.93%, respectively. However, their performance does not match that of LLaMA 3, GPT-4o, and DeepSeek-v3. Developers, classified into Cluster 4, exhibit the lowest median SRR, which may reflect that manual refactorings often prioritize functionality, maintainability, or project-specific goals over systematic code smell reduction. Core developers achieve an average of 2.0%, while peripheral developers experience an average of 2.5%, indicating more variability in impact. One-time contributors exhibit no measurable code smell reduction, likely due to unfamiliarity with the codebase. LLMs outperform all developer categories in code smell reduction, under the condition that the refactored code passes existing unit tests. However, this should not be interpreted as LLMs producing higher-quality refactorings overall. Developer refactorings may prioritize functionality, maintainability, or design goals not captured by SRR alone. Thus, LLMs complement rather than replace developer intent and expertise in real-world scenarios.

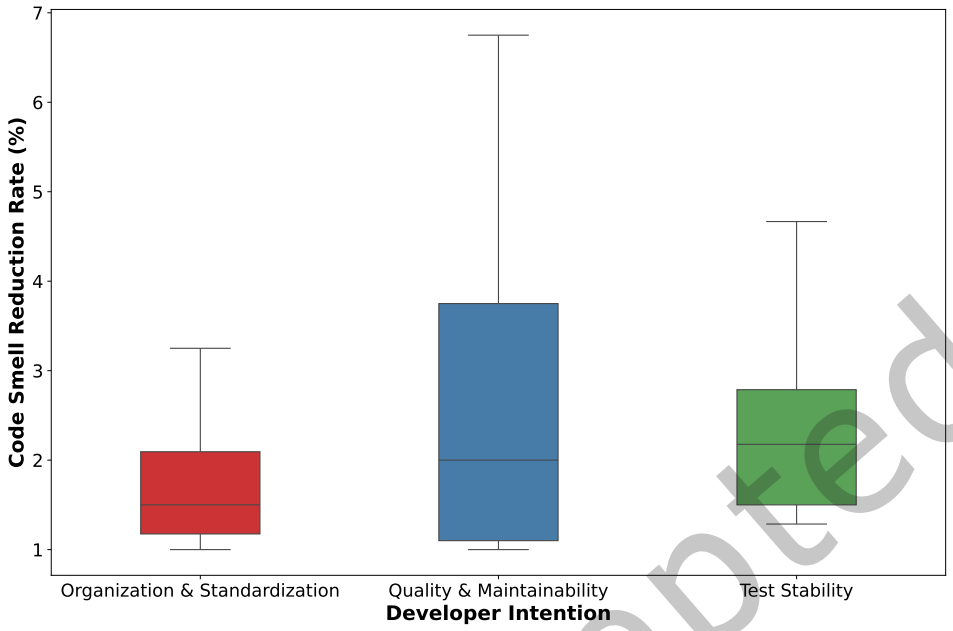

Fig. 8. Distribution of Code Smell Reduction Rates Across 30 Projects for Different Developer Refactoring Intentions.

Developer refactoring intentions are reflected in measurable improvements to code quality, as indicated by code smell reduction rates (SRR). As shown in Figure 8, commits aimed at improving code quality and maintainability achieve the highest average SRR with a median of 2.0% and several commits exceeding 5.0%, indicating that these refactorings more directly target and eliminate code smells. Refactorings motivated by code organization show slightly lower median SRR of 1.5%, likely due to their focus on structural consistency rather than semantic quality improvements. Commits associated with test stability exhibit a higher median SRR of 2.18%, suggesting that ensuring reliable and consistent tests contributes to the removal of some code smells. These results confirm that the effectiveness of developer refactorings in removing code smells is influenced by the underlying intention.

Summary of Results

LLM-generated refactorings can significantly improve code quality when they pass unit tests. Among the evaluated models, **GPT-4o** achieves the highest unit test pass@5 rate at **93.7%**, followed by **DeepSeek-v3** at **91.8%**, **GPT-4o Mini** at **88.0%**, **LLaMA 3** at **82.5%**, and **StarCoder2** at **49.6%**. In terms of code smell reduction, **LLaMA 3** achieves the highest median SRR at **15.15%**, followed by **GPT-4o** at **12.21%**, **DeepSeek-v3** at **10.42%**, **GPT-4o Mini** at **6.78%**, and **StarCoder2** at **4.93%**. These results show that LLMs can provide effective refactorings when combined with reliable unit testing.

4.2 RQ2: Which quality aspects of the code are most effectively reduced by LLMs?

4.2.1 ***Motivation.*** Quality refactorings are linked with the elimination of code smells [10, 70, 77]. These smells can range from simple syntactic issues, such as *Long Statement* and *Empty Catch Clause*, to more complex

design flaws, such as *Insufficient Modularization* and *Multifaceted Abstraction*. Understanding which types of code smells are most effectively reduced by developers or by LLMs is crucial for identifying their strengths and limitations. Developers have domain knowledge and an understanding of software design principles, while LLMs can apply their extensive training on code patterns and idioms to address repetitive and rule-based smells. Given that different types of code smells vary in complexity and context dependency, this research question aims to explore how well different LLMs and developers perform across removing various categories of code smells. By investigating which code smells each refactoring approach excels at reducing, we can better understand the strengths and weaknesses of LLMs in code refactoring.

4.2.2 ***Approach.*** To assess the effectiveness of LLMs in improving code quality metrics, we collect code metrics from three versions of the code using the static analysis tool, Understand [62]: (1) the unrefactored code, to assess its quality before refactoring; (2) the developer-refactored code, to evaluate its quality after developer refactoring; (3) and the LLM-refactored code, to assess its quality after LLM refactoring. This analysis allows us to systematically compare the effectiveness of refactorings carried out by the LLMs. We let A_{before} represent the value of the code metric attribute before refactoring, and A_{after} represent its value after refactoring. We calculate the code metrics improvement rate using the same formula as SRR:

$$\textit{Improvement Rate} = \frac{A_{\text{before}} - A_{\text{after}}}{A_{\text{before}}} \times 100 \tag{3}$$

Then, using the Scott-Knott Effect Size Difference (ESD) [73, 74] test on the distribution of the calculated improvement rates among code metrics, we examine the statistical difference in the quality of refactorings produced by the LLMs. For each of the 30 projects, we compute the code metric improvement percentages for each refactoring method. The Scott-Knott ESD test then partitions these distributions into ranked clusters.

It is also essential to consider whether the refactored code enhances readability and maintainability. Complexity and maintainability are closely linked; existing literature shows that reducing code complexity results in improved maintainability [4]. For instance, the Maintainability Index (MI) introduced by Coleman et al. [17] explicitly incorporates cyclomatic complexity, indicating that decreases in complexity directly enhance maintainability (MI):

$$\text{MI} = \max\left(0, \frac{171 - 5.2 \times \ln(\text{Halstead Volume}) - 0.23 \times (\text{Cyclomatic Complexity}) - 16.2 \times \ln(\text{Lines of Code})}{171} \times 100\right)$$

To assess readability, we sample 120 refactorings from each of StarCoder2, GPT-4, and LLaMA 3, totaling 360 (95% confidence and 5% margin of error). Two independent human evaluators (the first author and a collaborating master's candidate) then conduct a manual review, scoring each snippet on a 1 - 100 "ease of reading" scale. Independently, we apply an LLM-as-a-judge [92] to measure the readability of the code. To this end, we prompt o4-mini using the following zero-shot prompt template:

> On a scale from 1 (very difficult) to 100 (very easy), please rate the readability of this refactored code snippet. Justify your rating in one concise sentence.

For each model, we compute its mean readability score and standard deviation, and we measure alignment with human judgements via Pearson's r between o4-mini's ratings and the average manual scores.

To analyze the effectiveness of refactorings performed by developers and LLMs in reducing different types of code smells, we compare the performance of our selected LLMs (*i.e.*, StarCoder2, LLaMA 3, GPT-4o Mini, GPT-4o, and DeepSeek-v3) in code smell reductions and quantify the differences for each specific type of code smell. Specifically, we examine each commit that contains at least one instance of a code smell, ensuring meaningful

Table 6. Comparison of Improvement in Metrics (%) Over 30 Projects Between LLMs. Scott-Knott ESD rank is shown for the code quality metrics that are significantly improved by any LLM. A lower rank (e.g., 1) indicates a greater improvement of that metric.

	StarCoder2		**LLaMA 3**		**GPT-4o Mini**		**DeepSeek-v3**		**GPT-4o**	
Metric	Average	Rank	Average	Rank	Average	Rank	Average	Rank	Average	Rank
Coupling	19.9	2	20.5	2	21.4	1	21.9	1	**22.3**	1
Modularity	16.7	1	16.1	2	15.8	2	16.3	2	**16.9**	1
Cohesion	23.4	2	24.0	1	22.3	3	**24.2**	1	23.8	2
Complexity	16.5	3	17.2	2	17.9	1	18.4	1	**18.9**	1
Average	19.1	2	19.5	1	19.4	1	20.1	1	**20.5**	1

comparisons by focusing only on cases where a code smell is present in the original (*i.e.,* pre-refactoring) version. The detailed steps conducted are provided below:

Comparison of Differences: Given that our data does not follow a normal distribution, we apply the Scott-Knott ESD non-parametric test [73, 74] to partition the distributions of code smell reduction rates among the different LLMs and developers into statistically distinct groups. This method determines if one group tends to achieve higher reductions than another based on the ranks of the data points, but it also integrates effect size information to ensure that the differences are both statistically and practically meaningful. Each data point represents the reduction achieved for a specific code smell within a single project (*i.e.,* the reduction rate after refactoring by each LLM and the developers).

4.2.3 ***Findings.*** **LLMs consistently improve key code quality metrics related to cohesion, modularity, coupling, and complexity.** Table 6 summarizes the average improvements across 30 projects for each metric category. The comparison highlights StarCoder2, LLaMA 3, GPT-4o Mini, DeepSeek-v3, and GPT-4o, showing clear performance differences across models. Overall, the stronger models, particularly DeepSeek-v3 and GPT-4o, tend to achieve the highest Scott–Knott ranks.

LLaMA 3 achieves one of the strongest cohesion improvements with an average 24.0% gain at rank 1, reflecting its ability to consolidate related behaviors and reduce dispersion of responsibilities within classes [72, 88]. StarCoder2 also performs well, reaching **23.4%** at rank 2, while GPT-4o Mini improves cohesion by **22.3%**. DeepSeek-v3 slightly surpasses all models with a **24.2%** improvement at rank 1, and GPT-4o also performs strongly with **23.8%** at rank 2. These results suggest that cohesion improvements are a shared strength among the top-tier models, with LLaMA 3 and DeepSeek-v3 showing the most consistent changes. Future refactoring tools may benefit from leveraging these models' cohesion-centric transformations alongside developer oversight to better target scattered or weakly related logic.

StarCoder2 and GPT-4o achieve the strongest improvements in modularity. With an average improvement of **16.7%** and rank 1 in the Scott–Knott analysis, StarCoder2 is an effective model at reorganizing responsibilities and strengthening separation of concerns [56]. GPT-4o achieves **16.9%** improvement in modularity, placing it among the top performers for modularity restructuring. These results suggest that both code-specialized and general-purpose models are capable of redistributing logic, splitting responsibilities, and improving module boundaries, with StarCoder2 maintaining a notable edge likely due to its training on code-centric corpora [36, 41]. This highlights an opportunity to integrate such models into developer-facing tooling that proposes modularization strategies with strong structural precision.

GPT-4o achieves the most substantial reductions in code complexity. With an average improvement of **18.9%**, GPT-4o attains the highest Scott–Knott rank and demonstrates exceptional capability in simplifying control flow, reducing branching, and clarifying decision logic [43]. DeepSeek-v3 follows with a strong **18.4%** at rank 1, showing similar effectiveness in restructuring complex logic. GPT-4o Mini also performs well with **17.9%** at rank 1, followed by LLaMA 3 at **17.2%** and rank 2. StarCoder2 achieves **16.5%** at rank 3, reflecting solid but comparatively smaller reductions. These results indicate that larger and more reasoning-capable models excel at identifying redundant patterns and reorganizing nested logic. For practitioners, models such as GPT-4o and DeepSeek-v3 appear especially well-suited for tackling deeply nested branches and repetitive decision structures, with future work potentially examining whether these complexity reductions lead to lower defect-proneness in real systems.

Coupling reduction is strongest in the top-tier models, with GPT-4o achieving the highest improvement. GPT-4o achieves an average reduction of **22.3%**, placing it at rank 1 and demonstrating a strong ability to limit unnecessary dependencies between components. DeepSeek-v3 achieves **21.9%** improvement at rank 1, and GPT-4o Mini also performs strongly at **21.4%** at rank 1. LLaMA 3 improves coupling by **20.5%**, while StarCoder2 achieves **19.9%**. These results suggest that the newer and larger models (DeepSeek-v3 and GPT-4o) may incorporate stronger internal representations of design heuristics that favor local reasoning, reduced cross-class reliance, and more modular interactions. Coupling reduction remains an area where automated refactoring can provide substantial benefits, especially when combined with architectural review to ensure that dependency minimization aligns with project goals.

Table 7. Readability scores assigned by humans and by each LLM on 120 refactorings per model

	Human	StarCoder2	GPT-4	LLaMA 3
Mean score	71.2	69.5	72.1	68.8
Standard deviation	11.0	13.2	11.8	12.9
Median score	72.0	70.0	73.0	69.0
Pearson's *r* w/ human	–	0.78	0.84	0.75

Table 7 presents the results of the readability analysis for the three models. GPT-4 achieves the highest mean readability score (72.1), exhibits the tightest score distribution (SD = 11.8), and demonstrates the strongest alignment with reviewers' ratings (Pearson r = 0.84). This suggests that GPT-4's refactorings adhere most closely to common coding conventions, producing consistently clear and predictable code. StarCoder2 yields a mean score of 69.5 with a wider distribution (SD = 13.2) and a moderate correlation with manual judgments (r = 0.78), which may reflect occasional variability in naming consistency. LLaMA 3 attains a mean readability of 68.8 and SD = 12.9 (r = 0.75), possibly due to its smaller context window.

Overall, no single LLM dominates across all code quality dimensions. LLaMA 3 excels at improving cohesion, StarCoder2 at improving modularity, and GPT-4 at reducing complexity and coupling. This specialization suggests that automated refactoring tools could benefit from model selection strategies, where different LLMs are invoked depending on the refactoring goal. For example, a hybrid system might use StarCoder2 for improving modularity in data-heavy classes, while invoking GPT-4 for logic simplification in control-flow intensive code. Future work should explore ensemble approaches, multi-agent LLM architectures, or coordination frameworks that combine the strengths of multiple LLMs to maximize quality improvements across metrics.

LLaMA 3 is the most effective LLM in reducing all types of code smells detected by DesigniteJava, consistently outperforming other LLMs across each smell category. As illustrated in Table 8, LLaMA 3

Table 8. Comparison of Code Smell Reductions Among StarCoder2, GPT-4, Llama3, and Developers.

Design Code Smell	**Llama3**	**StarCoder2**	**GPT-4**	**Developers**
Broken Hierarchy	1	2	2	3
Broken Modularization	1	2	2	3
Cyclic-Dependent Modularization	1	2	2	3
Deficient Encapsulation	1	3	2	4
Imperative Abstraction	1	2	1	3
Insufficient Modularization	1	2	3	4
Missing Hierarchy	1	1	1	2
Unnecessary Abstraction	1	2	2	3
Unutilized Abstraction	1	2	2	3
Wide Hierarchy	1	1	1	2
Multifaceted Abstraction	1	2	2	3
Rebellious Hierarchy	1	1	2	2
Abstract Function Call From Constructor	1	1	1	2
Implementation Code Smell	**Llama3**	**StarCoder2**	**GPT-4**	**Developers**
Complex Conditional	1	3	2	4
Complex Method	1	2	1	3
Empty catch clause	1	2	1	3
Long Identifier	1	3	2	4
Long Method	1	2	1	3
Long Parameter List	1	2	2	3
Long Statement	1	2	3	4
Magic Number	1	2	2	3
Missing default	1	2	3	4

consistently receives a rank of 1 for every code smell type from the Scott-Knott ESD test, indicating that it is the most effective at reducing code smells across both implementation and design categories.

StarCoder2 performs better than GPT-4 in reducing *Insufficient Modularization, Rebellious Hierarchy, Long statement,* and *Missing Default*, as reflected by its lower Scott-Knott ranks in these categories. Specifically, StarCoder2 ranks second while GPT-4 ranks third for both *Insufficient Modularization* and *Missing Default*, and it achieves a rank of 2 in *Long Statement* compared to GPT-4's rank of 3. Additionally, StarCoder2 matches LLaMA 3 in *Rebellious Hierarchy* with a rank of 1, outperforming GPT-4, which ranks 2. These results suggest that StarCoder2 may be more effective at addressing structural and pattern-based code issues that require systematic transformations. Notably, its success in reducing *Long Statement* and *Missing Default* highlights its strength in improving code readability and completeness in control structures.

GPT-4 achieves a lower rank, and is more effective than StarCoder2 in addressing *Deficient Encapsulation, Imperative Abstraction, Complex Conditional, Complex Method, Empty Catch Clause, Long Identifier,* and *Long Method.* GPT-4 demonstrates superior performance in these smells, particularly those that require deeper reasoning about code structure and abstraction. The ability to reduce *Deficient Encapsulation* and *Imperative Abstraction* suggests that GPT-4 is better at handling design-related concerns that involve improving abstraction. Additionally, its effectiveness in reducing *Complex Conditional* and *Complex Method* indicates that GPT-4 has a stronger capability in restructuring intricate logic and improving method clarity.

As discussed in Section 3.4.1, although there is a relationship between refactoring and the elimination of code smells [23, 24, 48], not all code smells can be directly refactored in a way that can be captured by certain types of refactorings [1]. Certain code smells often require semantic modifications rather than structural refactorings, making them less likely to be formally linked with a set of refactoring types. For example, addressing the *God Class* smell typically involves breaking up a large class that centralizes too many responsibilities. This process may require a combination of refactorings such as *Extract Class*, *Move Method*, and *Introduce Interface*, applied iteratively and guided by domain knowledge. Since not all code smells are fixable through structural refactoring alone, we do not treat smell elimination as a strict requirement. However, we do not assume that every code smell is fully resolvable through classical refactoring operations. Some smells require semantic or architectural changes that go beyond the expressiveness of individual refactoring types. This choice acknowledges that some smells require semantic or architectural changes beyond classical refactoring, while still allowing us to quantify structural quality improvements made by developers and LLMs.

Summary of Results

Our findings show that the strongest models, GPT-4o, DeepSeek-v3, and LLaMA 3 consistently achieve the highest improvements across all static code quality metrics. GPT-4o achieves the best performance in reducing complexity with an **18.9%** improvement, followed closely by DeepSeek-v3 (**18.4%**) and GPT-4o Mini (**17.9%**). For cohesion, **DeepSeek-v3 obtains the highest improvement (24.2%)**, closely followed by LLaMA 3 (**24.0%**), making both models particularly strong at consolidating related responsibilities within classes. In modularity, StarCoder2 remains competitive with the highest improvement (**16.7%**), while GPT-4o (**16.9%**) and DeepSeek-v3 (**16.3%**) also perform strongly. GPT-4o shows the best reduction in coupling (**22.3%**), with DeepSeek-v3 (**21.9%**) and GPT-4o Mini (**21.4%**) close behind, indicating that these models are especially effective at minimizing unnecessary inter-class dependencies. LLaMA 3 is the most effective LLM for reducing all types of code smells, consistently achieving the best Scott-Knott rank across both implementation and design code smell categories. StarCoder2 is more effective for modularization-related and structured refactorings, while GPT-4 is better at reducing encapsulation-related and complex logic smells. These results highlight the varying strengths of different LLMs, with LLaMA 3 demonstrating superior overall effectiveness. Future research could explore leveraging the unique advantages of each model for a hybrid approach to automate code refactoring.

4.3 RQ3. Which refactoring types are most effective for improving code quality?

4.3.1 ***Motivation.*** Refactoring types refer to the fine-grained categories of refactoring, such as renaming variables for clarity, extracting methods to reduce code duplication, or reorganizing classes to better align with design principles. In this research question, we aim to understand how various refactoring types are applied to achieve quality improvements by GPT-4o Mini, LLaMA 3, and StarCoder2. This comparison allows us to determine whether these approaches prioritize different refactoring types and highlights their respective strengths in applying them. This information can reveal the strengths and weaknesses of LLMs in refactoring tasks, helping to identify which refactoring types should be prioritized in LLM-based refactoring tools and development practices.

4.3.2 ***Approach.*** To analyze the different types of refactorings applied and assess their impact on code quality, we first collect the refactorings performed by the LLMs and developers. We then analyze the statistical differences in refactoring types and their effects on code quality between the LLMs. The steps of this analysis are detailed below.

Quality and Impact Measurement: We use RMiner 3.0 [77] to extract all the refactorings applied by the LLMs and developers across all projects. RMiner is the most comprehensive refactoring detection tool for Java, capable of identifying up to 102 refactoring types, with a precision of 99.8% and a recall of 98.1% [77, 78]. Then, we examine the distribution of frequency of each refactoring type performed across all projects together to understand any differences in applying refactoring strategies between the LLMs. Next, we analyze whether the LLMs perform and prioritize different refactoring types. Finally, we assess the effectiveness of these refactorings by calculating the code smell and code quality metrics changes for each refactoring type applied by developers and LLMs.

While some code smells correspond directly to traditional refactoring types (e.g., using the *Extract Method* refactoring to address the *Long Method* smell by splitting lengthy methods into smaller methods, or applying the *Move Method* refactoring to resolve the *Feature Envy* smell by relocating methods closer to the data they use), others involve semantic modifications that do not fit within RMiner's classification. For example:

- **Empty Catch Clause**: This issue is typically resolved by either removing the empty catch block if it serves no purpose or adding proper exception handling logic. Such corrective actions, although beneficial, are generally considered bug fixes or code corrections rather than formal refactoring types.
- **Missing Default Case in Switch Statements**: Fixing this requires adding a default case in a switch statement to handle unspecified values. This is a logic enhancement rather than a structural transformation.
- **Unnecessary Abstraction**: Resolving this often involves removing redundant interfaces or abstract classes when they add no meaningful value. However, such removals are not always categorized as formal refactorings, as refactoring implies restructuring the existing code to improve design without changing external behavior. Eliminating unnecessary abstractions can sometimes alter the external API or the inheritance structure, thus extending beyond the conventional definition of refactoring.

Significance and Rank-Based Comparison: To assess whether the types of refactoring performed by each LLM lead to statistically distinct improvements in quality, we employ the Scott–Knott ESD test [73, 74] to both code smell reduction and code metric improvement data. For code smells, each data point represents the total reduction in smells achieved by a specific refactoring type executed by a given LLM. The Scott–Knott ESD test identifies whether the distributions of smell reductions differ significantly across LLMs for each refactoring type and assigns them into ranked performance clusters accordingly. We apply the same procedure to analyze improvements in code quality metrics, allowing us to consistently compare LLM effectiveness across both aspects of code quality: code smells and code quality metrics.

4.3.3 ***Findings.*** **GPT-4 consistently performs more structural refactorings than both StarCoder2 and LLaMA 3 across projects.** Figure 9 shows that GPT-4 has the highest median counts for structural refactorings such as *Extract Method* and *Inline Variable*. StarCoder2 typically performs more *Move Method* refactorings than GPT-4 and LLaMA 3. LLaMA 3 the highest median counts for *Change Attribute Access Modifier* and *Remove Method Annotation*. These patterns indicate clear behavioral differences across models, with GPT-4 being the most aggressive in applying structural code transformations.

Table 9 presents the Scott–Knott ESD ranks for different refactoring types based on their effect on code smell reduction rates, as achieved by StarCoder2, GPT-4, and LLaMA 3. The table shows that the effectiveness of each LLM varies depending on the type of refactoring being performed. Some LLMs are more effective than others for specific refactoring operations. For example, for *Add Attribute Annotation*, both StarCoder2 and GPT-4 achieve the first rank, outperforming LLaMA 3, which ranks second. Conversely, for *Add Parameter Modifier*, LLaMA 3 achieves the highest effectiveness (rank 1), followed by StarCoder2 (rank 2) and GPT-4 (rank 3). To distinguish the strengths of each LLM, we summarize key observations as follows:

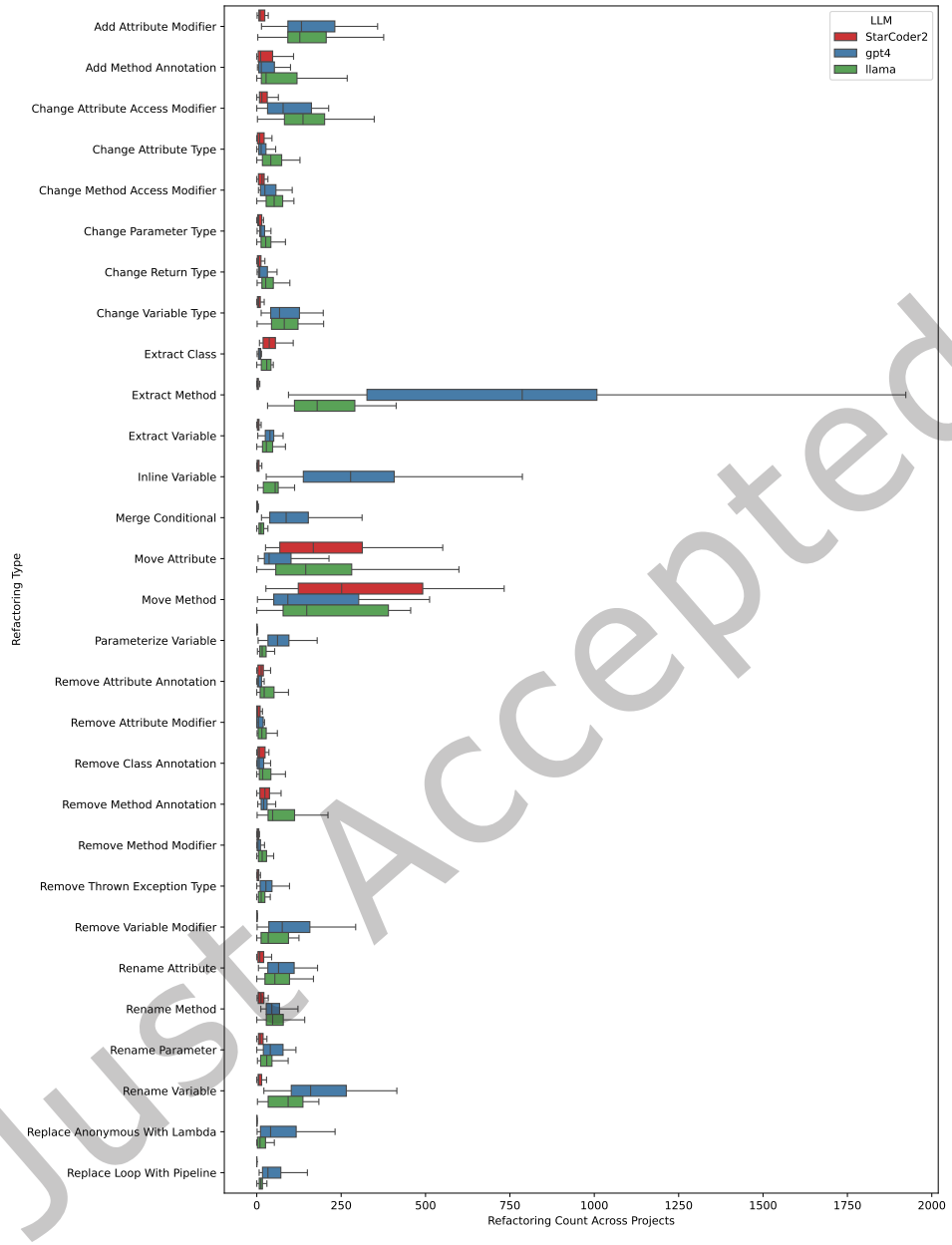

Fig. 9. Distribution of counts for refactoring types with differing median frequencies (i.e., range > 10) across StarCoder2, GPT-4, and LLaMA 3 in 30 projects

Table 9. Scott–Knott ESD ranks for comparison of different refactoring types' effect on code smell reduction between LLMs

Refactoring Type	StarCoder2	GPT-4	Llama 3
Add Attribute Annotation	1	1	2
Add Class Annotation	2	1	1
Add Class Modifier	2	1	1
Add Method Annotation	1	2	1
Add Method Modifier	2	2	1
Add Parameter Annotation	2	2	1
Add Parameter Modifier	2	3	1
Add Parameter	1	2	2
Add Thrown Exception Type	2	1	1
Add Variable Annotation	1	2	2
Add Variable Modifier	2	1	3
Change Attribute Type	2	2	1
Change Class Access Modifier	1	2	2
Change Parameter Type	2	2	1
Change Return Type	2	2	1
Change Thrown Exception Type	1	2	1
Change Type Declaration Kind	2	2	1
Change Variable Type	2	2	1
Encapsulate Attribute	2	1	2
Extract And Move Method	1	1	2
Extract Class	1	2	1
Extract Method	1	2	2
Extract Superclass	2	1	1
Inline Method	2	3	1
Inline Variable	1	2	2
Invert Condition	2	1	1
Localize Parameter	2	2	1
Merge Attribute	2	1	3
Merge Catch	3	2	1
Merge Conditional	3	2	1
Modify Class Annotation	2	2	1
Modify Parameter Annotation	3	2	1
Move And Inline Method	3	2	1
Move And Rename Attribute	3	2	1
Move And Rename Class	2	2	1
Move And Rename Method	3	2	1
Move Attribute	1	2	1
Move Class	2	2	1
Move Method	1	2	1
Parameterize Attribute	1	2	3
Parameterize Variable	1	3	2
Pull Up Attribute	1	1	2
Pull Up Method	1	2	1
Push Down Attribute	2	1	2
Push Down Method	2	2	1
Remove Attribute Annotation	2	1	2
Remove Class Annotation	2	2	1
Remove Class Modifier	2	2	1
Remove Method Annotation	2	2	1
Remove Method Modifier	3	2	1
Remove Parameter Annotation	2	1	1
Remove Parameter Modifier	2	1	1
Remove Parameter	1	2	1
Remove Thrown Exception Type	1	2	1
Remove Variable Annotation	2	1	1
Remove Variable Modifier	1	2	1
Rename Attribute	1	2	1
Rename Method	1	2	1
Rename Parameter	1	2	2
Rename Variable	1	2	1
Reorder Parameter	3	2	1
Replace Anonymous With Class	1	2	1
Replace Anonymous With Lambda	3	2	1
Replace Attribute With Variable	1	1	2
Replace Loop With Pipeline	1	2	1
Replace Variable With Attribute	1	2	1
Split Conditional	2	2	1
Split Parameter	3	1	2

- StarCoder2 tends to perform best on fine-grained structural refactorings involving clear local changes, such as: *Extract Method*, *Inline Variable*, *Rename Parameter*, and *Replace Loop With Pipeline*.
- GPT-4 shows the best performance on refactorings that add or modify semantic metadata or annotations, including: *Add Class Annotation*, *Add Variable Modifier*, *Encapsulate Attribute*, and *Merge Attribute*.

Table 10. Scott–Knott ESD Ranks for Code Metrics Improvement by Refactoring Type

Refactoring Type	StarCoder2	Llama3	GPT-4	Developer
Rename Class	3	2	2	1
Encapsulate Attribute	2	3	2	1
Change Return Type	2	1	3	2
Change Attribute Access Modifier	3	1	2	2
Extract Method	2	2	1	1
Inline Variable	3	1	3	2
Move Method	3	1	3	2
Rename Variable	2	1	3	3

- LLaMA 3 consistently outperforms the other models across most refactoring types, particularly complex operations that impact structure, control-flow, or method composition, such as: *Inline Method*, *Merge Conditional*, *Extract Superclass*, *Change Return Type*, and *Push Down Method*.

Our results highlight that developers excel in refactorings requiring deep, context-sensitive knowledge, whereas LLaMA3 consistently outperforms GPT-4 and StarCoder2 in refactoring tasks involving precise, localized code transformations. Table 10 shows the Scott–Knott ESD ranks for different refactoring types based on their effect on code metric improvement rates, as achieved by StarCoder2, GPT-4, and LLaMA 3, and developers. Developers achieve the highest performance (rank 1) in complex, structural refactorings such as *Rename Class* and *Encapsulate Attribute*, reflecting their nuanced understanding of the underlying design and intent of the codebase. These refactoring types often require a detailed comprehension of code architecture and structural dependencies, areas in which developers consistently excel. Among the LLMs, LLaMA3 demonstrates superior performance for several specific refactoring operations that typically involve targeted adjustments to code structure and semantics, such as *Change Return Type*, *Change Attribute Access Modifier*, *Inline Variable*, *Move Method*, and *Rename Variable*, consistently securing the top rank (1). These refactorings benefit from LLaMA3's ability to handle code transformations that directly influence code metrics, particularly around complexity and coupling. GPT-4 generally achieves moderate performance (rank 2 or 3), indicating a balanced capability without excelling distinctly in any specific code metric or structural transformation. StarCoder2 often ranks lower (rank 3) but shows relatively better performance in automated, rule-based refactorings aimed at visibility and encapsulation, such as *Change Method Access Modifier* and *Encapsulate Attribute*, reflecting its syntactic transformation strengths. These distinct performance patterns suggest that LLMs, especially LLaMA3, can provide substantial value in automating metric-driven code improvements.

Although refactoring improves maintainability, developers often avoid built-in tools due to concerns over lost control, trust, and workflow disruption [46, 59]. However, Previous work shows that in situ AI assistants achieve higher adoption and satisfaction than manual tool invocation [34, 42]. Therefore, by interleaving LLM refactoring suggestions directly into the IDE [30] rather than forcing developers to invoke a separate refactoring command, an LLM-assisted approach preserves developer agency and reduces friction. The developer may initiate the process by selecting a "Suggest Refactoring" option within the IDE, which subsequently invokes the LLM to generate a candidate refactoring. This candidate is then automatically validated against the existing suite of unit tests, and only if it passes all tests is the refactoring presented to the developer for consideration.

Summary of Results

Our analysis shows that the effectiveness of refactoring varies considerably by task. For code smell reduction, both StarCoder2 and GPT-4 tend to lead in refactorings like *Add Attribute Annotation*, and different rank orderings are observed for other types. In contrast, for code metrics improvement, developers outperform the LLMs on context-sensitive refactorings such as *Rename Class* and *Encapsulate Attribute*. Notably, LLaMA3 excels in automated transformation tasks like *Change Return Type*, *Inline Variable*, and *Move Method*. These results highlight that while LLMs are well-suited for syntactic, rule-based changes, developers maintain an edge in refactorings that require deeper contextual understanding. A combined approach leveraging the strengths of both LLMs and human expertise could thus yield optimal code quality improvements.

4.4 RQ4. How does prompt engineering affect the quality of LLM-generated refactorings?

4.4.1 ***Motivation.*** The limited application of the full range of refactoring types by LLMs results in coverage gaps in certain refactoring types. Although developers perform a broad spectrum of refactoring operations spanning organizational, maintainability, and test-stability refactorings, LLMs concentrate on only a handful of structural changes. As RQ3 showed, the LLMs rarely apply several high-impact refactorings (e.g., Encapsulate Attribute, Extract Superclass, Push Down Method) that developers routinely use. This uneven distribution means that LLM-driven refactorings leave many code smells and architectural improvements unaddressed, limiting the practical benefit of fully automated code maintenance. In this research question, we aim to expand the capabilities of LLMs in refactoring at the prompting level [90], which involves designing the input prompts to guide the LLM responses more effectively; we aim to enhance the ability of our selected LLMs to perform more complex and a wider variety of refactoring types. More specifically, we want to enable LLMs to perform refactoring types that they did not perform in the third research question or perform less frequently compared to developers. The objective is to determine whether modifying the instructions or providing additional context in the prompt can encourage the LLMs to execute more complex refactoring types, typically performed by developers, thus improving the code quality.

4.4.2 ***Approach.*** To enhance the refactoring experience of the LLMs, we aim to evaluate different prompting techniques and identify the most effective ones for improving code quality. In the following, we describe our approach in detail.

Prompt strategy selection: Refactoring real-world, multi-file commits requires prompting techniques that can provide structural guidance while remaining within token limits. We therefore restrict our evaluation to three prompting strategies that are well-suited to this setting and widely used for transformation tasks. Zero-shot prompting establishes the model's baseline capability without additional guidance. Chain-of-thought prompting is appropriate when the goal is to expose the model to developer-style refactoring reasoning, allowing it to articulate why certain refactorings should be applied. One-shot RAG prompting is suitable for structural refactoring because it gives the model a concrete example of a developer-performed transformation, enabling it to infer transformation patterns directly from code rather than from natural-language descriptions. Other prompting techniques, such as few-shot prompting, and template chaining, are considered but are impractical to use in our selected models due to the large size of multi-file refactorings and the limited context windows of the evaluated models. Thus, the selected prompting strategies represent the most appropriate balance between guidance, feasibility, and model constraints for the task of multi-file refactoring. We first identify refactoring types that are consistently performed by developers but are missing in the LLMs' outputs as the results from RQ3 (Section 4.3.3). Once we

identify these refactoring types, we design specific prompts to guide the LLMs in applying them. These prompts are categorized as follows:

- Zero-shot Prompt (baseline): This prompt is the baseline prompt used in RQ1, as shown in Figure 5.
- Chain-of-thought: Shown in Figure 10, this prompt provides suggestions for refactorings, along with the explanation of that refactoring type, by including common refactoring patterns developers perform that the LLM does not. For example, the prompt may instruct, "Consider using Extract Method or Inline Method," guiding our LLMs toward specific types of refactorings that developers perform with more efficiency.
- One-Shot RAG Prompt: Shown in Figure 11, in this prompting approach, we provide an example of a correctly refactored code snippet by the developer to the LLMs along with the code to be refactored. This technique guides the model to learn from the provided example and apply the learned pattern to the code that needs refactoring. For instance, the prompt would present a refactored code snippet where a method has been successfully extracted and then ask the model to perform similar refactorings on a new code sample. We perform one-shot prompting and not a few-shot prompting due to the context window limitations of StarCoder2 and LLaMA 3 (*i.e.* 16,384 tokens and 8,192 tokens, respectively). Since each example in the prompt can be several thousand tokens long, especially for multi-file or class-level refactorings, incorporating more than one example risks exceeding the LLMs' prompt capacity or causing token overflow. Even when within the token limit, longer prompts may dilute the LLMs' focus and degrade performance [40]. Thus, rather than maximizing the number of examples, our method prioritizes the relevance of a single high-quality exemplar.
  Selecting a relevant example requires retrieving a prior instance where a comparable refactoring type is applied to semantically similar code. To accomplish this, we use RMiner [77] to extract refactoring operations from the full commit history of the project and determine the precise refactoring type performed by the developer (e.g., *Extract Method*, *Move Method*, *Inline Method*, *Extract Class*). This ensures that the exemplar provided to the LLM aligns with the intended structural transformation. Furthermore, to identify the most appropriate example within that refactoring category, we compute embeddings of the pre-refactoring code using Microsoft's CodeBERT-base [32]. These embeddings capture structural and semantic similarity, enabling fine-grained matching beyond simple textual or syntactic overlap. We index all such embeddings using FAISS (*i.e.*, Facebook's vector similarity search library) [31] and perform a nearest-neighbor search restricted to commits that share the same RMiner-identified refactoring type.
  Specifically, for the retrieved commit, we extract the code *before* the refactoring (retrieved via embedding search) and the corresponding developer-produced *after* version that reflects the actual refactoring applied in that commit. This pair serves as the one-shot example in the prompt. In cases where the top retrieved example falls below a similarity threshold or no sufficiently close match exists, we fall back to a canonical template example for that refactoring type to ensure consistent guidance across cases.

Quality impact evaluation: To evaluate the impact of different prompting techniques, we compare the quality of refactorings generated by the LLMs across the unit test pass rate, which measures functional correctness and compilability, the code smell reduction rate, and code quality metrics. For each commit in our dataset, we generate refactorings using the 2 categories of prompts: one-shot RAG prompting and chain-of-thought prompting. We evaluate the effectiveness of these prompts by comparing their ability to improve code quality metrics and reduce code smells, with particular attention to whether the LLMs perform refactorings that they previously did not perform. To compare different prompting techniques and assess whether the prompting techniques lead to statistically significant improvements, we perform the Scott-Knott ESD test [73, 74] across the three distributions: zero-shot, one-shot, and chain-of-thought for each LLM. The data samples consist of quality improvements (*i.e.*,

Instruction
You are a powerful model specialized in refactoring Java code. Code refactoring is the process of improving the internal structure, readability, and maintainability of a software codebase without altering its external behavior or functionality. You must output a refactored version of the code. Explain the steps you took to refactor the code and why you selected the refactoring type/types you did.

Prompt
 # Suggested refactoring types:
{refactorings developers performed on this commit with definitions}
 # unrefactored code snippet(java):
{code_segment_before_refactoring}

 # refactored version of the same code snippet:

Fig. 10. Chain-of-Thought Prompt Used to Instruct StarCoder2 to Conduct Refactoring

Instruction
You are a powerful model specialized in refactoring Java code. Code refactoring is the process of improving the internal structure, readability, and maintainability of a software codebase without altering its external behavior or functionality. You must output a refactored version of the code.

Prompt
 # unrefactored code snippet(java):
{code_segment_before_refactoring}
 # refactored version of the same code snippet:
{developer_code_after_refactoring}

 # unrefactored code snippet(java):
{code_segment_before_refactoring}

 # refactored version of the same code snippet:

Fig. 11. One-shot Prompt Used to Instruct StarCoder2 to Conduct Refactoring

Table 11. Scott-Knott Test Results for Code Smell Reduction Rate (SRR) and Unit Test Pass Rate Across Different Prompting Techniques. The Scott-Knott (SK) ranking applies to SRR.

StarCoder2				
Prompting Method	**pass@1**	**SRR Median**	**SRR Average**	**SK Rank**
Zero-shot	33.4%	5.12%	6.72%	2
One-shot	33.7%	4.98%	6.55%	3
Chain-of-Thought	35.1%	5.45%	7.05%	1
Llama3				
Prompting Method	**pass@1**	**SRR Median**	**SRR Average**	**SK Rank**
Zero-shot	73.4%	15.24%	15.82%	1
One-shot	73.9%	14.98%	15.42%	2
Chain-of-Thought	74.7%	15.05%	15.66%	2
GPT-4				
Prompting Method	**pass@1**	**SRR Median**	**SRR Average**	**SK Rank**
Zero-shot	78.7%	6.82%	7.12%	2
One-shot	79.3%	6.75%	7.05%	2
Chain-of-Thought	80.1%	6.90%	7.20%	1

code smell reduction rate, and percentage of improvement in code metrics) achieved by LLMs under different prompt conditions for each project.

4.4.3 ***Findings.*** **Applying one-shot RAG prompting and chain-of-thought have more significant influences on code smell reduction than applying zero-shot prompting.** Table 11 summarizes the results for the unit test pass rates and code smell reduction rates across the 3 prompting methods: zero-shot, one-shot, and chain-of-thought prompts. Table 12 shows the results for code quality metric improvement rates across the 3 prompting methods.

- **One-shot RAG prompting** exhibits a trade-off compared to the zero-shot baseline. For StarCoder2, it improves unit test pass rates from 33.4% to **33.7%**, but results in a lower code smell reduction rate (SRR) of **6.55%** (Rank 3) compared to the SRR of zero shot at **6.72%** (rank 2). Similarly, for LLaMA3 and GPT-4, one-shot prompting slightly increases test pass rates but decreases SRR to **15.42%** and **7.05%**, respectively.

Table 12. Average Reduction in Code Metrics (%) by Prompting Method for Each LLM

LLM	Metric	Zero-shot		One-shot		Chain-of-Thought	
		Average	Rank	Average	Rank	Average	Rank
StarCoder2	Coupling	19.85	2	19.50	2	20.20	1
	Modularity	16.70	1	16.53	1	16.97	1
	Complexity	16.50	1	16.30	1	16.80	1
	Cohesion	23.35	2	23.10	1	23.75	1
Llama 3	Coupling	20.45	1	20.10	1	20.75	1
	Modularity	16.13	1	15.83	1	16.47	1
	Complexity	17.20	1	17.00	2	17.60	1
	Cohesion	24.00	2	23.65	2	24.35	1
GPT-4	Coupling	21.35	1	21.00	1	21.70	1
	Modularity	15.83	1	15.57	1	16.07	2
	Complexity	17.90	2	17.70	2	18.10	1
	Cohesion	22.30	3	22.05	2	22.55	1

While one-shot prompting performs worse in improving structural metrics, it enables a greater number of successful and compilable refactorings that pass tests. Overall, one-shot prompting sacrifices metric-based quality improvements in favor of generating more functionally correct and diverse refactorings, making it useful when increasing coverage and variety of correct outputs is prioritized over structural optimization.

- **Chain-of-Thought prompting** StarCoder2 and GPT-4 achieve the best performance. StarCoder2 achieved the highest unit test pass rate of **35.1%** and an SRR of **7.05%** (Rank 1). GPT-4 also reached its best pass rate at **80.1%** and an SRR of **7.20%** (Rank 1). For LLaMA3, chain-of-thought maintains a pass rate (**74.7%**) and SRR (**15.66%**, Rank 2). This method improved cyclomatic complexity more than other prompting methods, indicating its effectiveness in enhancing code readability and maintainability. Chain-of-Thought prompting thus appears particularly beneficial in tasks requiring deeper reasoning and structured guidance.
- **Zero-shot prompting** generally yielded baseline performance across all models. For StarCoder2, zero-shot achieves a unit test pass rate of **33.4%** and an SRR of **6.72%** (Rank 2). LLaMA3 shows better results, with a pass rate of **73.4%** and the highest SRR of **15.82%** (Rank 1). GPT-4 achieved a pass rate of **78.7%** and an SRR of **7.12%** (Rank 2). In terms of code quality metrics, zero-shot prompting outperformed one-shot prompting across nearly all models and metrics. For example, zero-shot prompting ranked first in **modularity and complexity improvement for all LLMs**, and also achieved rank 1 in **coupling improvement** for both LLaMA3 and GPT-4. In contrast, one-shot prompting ranked lower in complexity for LLaMA3 (rank 2), and lower in cohesion for GPT-4 (rank 2 vs. 3 in zero-shot). However, zero-shot prompting performs worse than chain-of-thought prompting in several cases, most notably in cohesion, where chain-of-thought achieves rank 1 for all three models, and in complexity for GPT-4 (rank 1 for CoT vs. rank 2 for zero-shot). Overall, zero-shot prompting provides a strong balance of structural quality improvements and prompting effort, outperforming one-shot in metric-based refactoring impact, but falling short of the higher semantic alignment seen with chain-of-thought prompting. It is well-suited for scenarios where minimal prompting is preferred without sacrificing code quality.

Overall, our findings highlight that while zero-shot provides a solid baseline, chain-of-thought prompting significantly enhances the refactoring performance, especially for StarCoder2 and GPT-4, indicating the effectiveness of structured reasoning in automated code refactoring tasks. LLaMA 3 may not experience an increase in refactoring performance with chain-of-thought prompting due to its default reasoning capabilities.

Summary of Results

Our findings highlight the importance of prompt design in leveraging the full potential of LLMs for code refactoring tasks. Incorporating examples (*i.e.*, one-shot RAG prompt) or providing more explicit instructions (*i.e.*, chain-of-thought prompt) improves both the functional correctness and overall quality of the refactorings generated by the LLMs. The variation in code smell reduction rates and unit test pass rates across different prompting methods underscores the value of prompt engineering in optimizing LLM performance for complex tasks like refactoring.

5 Threats to Validity

In this section, we outline potential threats to the validity of our study and the measures taken to mitigate them.

Threats to Internal Validity. Specify the validity of the methods applied in our study. While the performance of StarCoder2 could theoretically be influenced by specific hardware and software configurations, we ensured that all experiments were conducted under identical conditions across the same server, using consistent hardware and software setups. Therefore, running the experiments on a different server should not significantly affect the results, as long as comparable computational resources are used. This mitigates the concern of hardware or server variability as a threat to internal validity.

A potential threat to internal validity is the occurrence of model hallucination, where LLMs may generate code that appears correct but is not logically or syntactically valid. Such hallucinations could alter the accuracy of our results, particularly in evaluating unit test pass rates and refactoring effectiveness. We account for this by verifying the output through unit test evaluations and inspecting for invalid code, but hallucinations remain an inherent risk in LLM-generated code. While passing all unit tests helps filter out many hallucinated or semantically incorrect transformations, unit tests cannot provide a formal guarantee of semantic equivalence, and act as a proxy in large empirical settings, where manual involvement is not feasible. If test coverage is incomplete, some behavioral regressions or logic changes introduced by the LLM may remain undetected. Therefore, although our test-based validation reduces the risk of accepting incorrect refactorings, it does not fully eliminate this threat.

We utilize a randomly selected set of representative commits from our projects. However, our results may be influenced by the random selection of commits, as variations in the project settings or the choice of different subsets of commits could lead to differing outcomes. Some of the refactorings might span multiple commits, however, our approach focuses on analyzing refactorings within a single commit. This limitation could result in missing refactorings that are spread over several commits, potentially underestimating the complexity or scope of certain changes.

Another threat to internal validity concerns the purity of the refactoring commits in our dataset. While we use RMiner to identify refactoring operations and require that the refactored versions pass the original unit tests, as refactorings may be interleaved with bug fixes or feature updates. While our filtration minimizes the likelihood of semantic-altering changes, it cannot fully rule them out. Task-oriented tools such as Mantra [86] propose purity checkers to more precisely filter refactoring-only commits. Consequently, some degree of impurity may remain in our dataset and could influence the observed results. Future work can benefit from such purity checking tools for filtering refactoring-only commits.

While we include an analysis with StarCoder2 on repositories excluded from its training corpus to provide partial evidence that performance is not primarily driven by memorization or training data leakage, this does not constitute proof that larger, closed-source models are unaffected by such effects. In particular, the fact that a smaller model (StarCoder2) shows only marginal differences between seen and unseen projects does not scientifically rule out that larger models, which likely have higher capacity for pattern recall and high-probability completion, may benefit from having seen similar code or even the evaluated projects during training. The

training data for GPT-4 and LLaMA 3 are not transparent, and their higher capacity may increase the likelihood of recalling previously seen code patterns or repositories. Consequently, we cannot rule out that some portion of their performance may be influenced by prior exposure. We therefore avoid generalizing the StarCoder2 findings to these models and acknowledge this limitation when interpreting the results for closed-source LLMs.

Threats to External Validity. Indicate the generalizability of our approach. Our experiments are conducted on 30 open-source Java projects. Therefore, the findings may not be directly applicable to projects in other programming languages or domains. We make the replication package publicly available to allow other researchers to test across a more diverse set of projects and languages. While our results reflect the capabilities of StarCoder2, Llama 3, and GPT-4 LLMs, our evaluation approach is designed to be adaptable. Nevertheless, different LLMs or different versions of StarCoder may generate different results. Due to resource limitations and the long inference time of the LLMs (approximately eight minutes per commit), we could not scale our experiments to a larger dataset. Therefore, to avoid bias and ensure coverage while reducing the size of our dataset, we apply a systematic project selection and statistical commit sampling approach to our initial dataset, selecting a representative subset of commits and projects. We sample commits using 95% confidence and a 5% margin of error, which has been used by previous studies [48, 65, 83]. Our dataset size remains comparable with those used in prior LLM-based refactoring and code generation studies [20, 48, 58, 69, 91].

Threats to Construct Validity. Concern about our feature selection and our data preparation. We primarily rely on code smell reduction, static code quality metrics improvement, and unit test pass rates as indicators of refactoring quality. However, these metrics may not fully capture all dimensions of code quality, such as readability, performance, or future evolution. This limitation may affect the robustness of our conclusions across a broader range of quality indicators. We use RMiner [77], DesigniteJava [67], and the Understand tool [62] to gather our metrics for analysis, which are the best in the state of practice for Java; however, different tools may produce slightly different results.

In portions of our analysis (e.g., RQ3), we focus solely on comparing the relative performance of different LLMs rather than including developers' intent. This is because developers often perform refactorings with diverse and context-specific goals, such as improving maintainability in targeted areas or reducing technical debts under time constraints, or aligning with project-specific architectural decisions, that do not necessarily aim to reduce code smells or optimize all quality metrics simultaneously. Including developers in every smell-level or metric-level comparison could introduce misleading interpretations, especially in cases where their intent is not aligned with the metric being evaluated. As a result, our evaluation focuses on objective, outcome-based quality indicators rather than attempting to infer developer intent.

In scenarios where developers outperform LLMs, this advantage is often attributable to the broader context available to human contributors, a known limitation of LLM-based approaches that has been highlighted in prior studies.

Our study uses refactoring types that can be detected by RMiner [77], which includes the standard refactoring practices defined by Fowler [24]. RefactoringMiner [77] is a state-of-the-art refactoring detection tool that can detect 102 different refactoring types. However, developers can restructure code to adhere to project-specific design conventions, or prepare for future evolution. Such intents are typically not encoded in the refactorings detected by RMiner and therefore are not considered by our study. Our evaluation therefore captures the quality of LLM refactorings within the domain of structural, tool-detectable refactoring actions, rather than the full spectrum of human development goals.

Although RMiner allows us to capture explicit structural refactorings performed by developers, these commits may not always reveal the broader motivations behind each change (e.g., time pressure, risk mitigation, or compatibility constraints). We have examined the refactorings in our dataset and have not found evidence that developers make partial or incomplete transformations in our selected dataset. Instead, all of the commits contain localized, well-formed refactorings that map cleanly onto RMiner's taxonomy. Nonetheless, our comparison

focuses on the observable structural outcome rather than the full cognitive process of the developer. Therefore, our results do not account for implicit project-specific intent or long-term architectural planning that may influence human refactoring decisions but are not visible in the code diff.

6 Related Work

The application of Large Language Models (LLMs) in software engineering, particularly in code refactoring, has garnered significant attention. In this section, we categorize the related work into four key areas: (1) traditional code generation and refactoring methods, (2) code generation approaches before and after the rise of LLMs, (3) code refactoring using LLMs, and (4) prompt engineering and fine-tuning techniques to enhance LLM performance.

6.1 Traditional Code Generation and Refactoring Methods

Before the introduction of LLMs, code refactoring was primarily a manual process guided by developers. Fowler [24] provides a catalog of refactoring techniques, including *Extract Method*, *Move Method/Field, and Rename Method/Field*. These techniques have formed the foundation for guiding developers to improve code quality systematically. Furthermore, with the introduction of IDEs, developers gained access to built-in static refactoring tools. JRefactory [8] and Eclipse's Refactoring Engine [71] rely on static and rule-based tools to apply common refactorings, such as renaming methods and extracting code fragments. The IDE's static refactoring methods are effective for low-level refactoring tasks, which involve moving or renaming parts of code (e.g., rename methods and extract method); however, they require substantial manual intervention of developers to perform high-level refactorings, which demand deeper semantic understanding and decision-making (e.g., extract superclass and collapse hierarchy). Similar to refactoring, automatic code generation and repair primarily relied on rule-based systems and traditional program synthesis techniques, which use formal methods and predefined rules to generate code, limiting the ability to handle complex programming tasks. DeepCoder [3] synthesizes programs from input-output pairs, but struggles with complex coding tasks. AlphaCode [37] generates code from natural language descriptions but has limited generalization due to its training on programming competition problems. Our work expands on the previous static refactoring assistants and evaluates the LLM's ability to perform refactoring tasks on real-world commits.

6.2 Code Generation Approaches in the LLM Era

With the introduction of LLMs, a new opportunity for code generation has emerged. Codex [22] has demonstrated the potential of LLMs to generate code through natural language prompts. Codex has been evaluated for its ability to repair programs using test-based repair tools, showcasing strengths in code completion but also highlighting limitations in handling complex tasks due to a lack of semantic understanding. Xu et al. [84] explore the capabilities of LLMs in software engineering tasks, providing a systematic evaluation of models, such as Codex. Their work emphasizes the strengths of LLMs in automating code generation tasks, while also pointing out the limitations of these models in understanding deeper program semantics. Chen et al. [13] propose CodeT, which uses LLMs for code generation by proposing a method that automatically generates test cases to evaluate and select the best code samples. Shen et al. [68] introduce PanGu-Coder2, which applies an RRTF (Rank Responses to align Test and Teacher Feedback) framework to improve code generation performance. PanGu-Coder outperforms previous code LLMs such as AlphaCode [37] on CoderEval [89] and LeetCode benchmarks. In this work, we focus on the capabilities of LLMs in performing refactoring, which improves the internal quality of the existing code rather than generating new code. We measure the performance of LLMs by evaluating the unit test pass rate, the reduction of code smells, and improvements in code quality metrics, which have been largely unexamined in prior research.

6.3 Code Refactoring Using LLMs

LLMs have been adopted for code refactoring tasks. Shirafuji et al. [69] demonstrate that LLM-generated refactorings can lead to significant improvements in cyclomatic complexity and a decrease in the average number of lines of code (LOC). By generating 10 candidate solutions (pass@10) with GPT-3.5 on each programming problem in their dataset, where the average LOC is 14.8, they find that 95.68% of programs could be successfully refactored. Choi et al. [15] analyze 17 open-source projects, focusing on methods with high cyclomatic complexity and iteratively applying LLM refactoring to reduce cyclomatic complexity. They find that the proposed LLM refactorings can reduce the average cyclomatic complexity by up to 10.4% within 20 iterations. Pomian et al. [58] propose EM-Assist, an IntelliJ IDEA plugin that uses LLMs to generate *Extract Method* refactoring suggestions. EM-Assist validates, enhances, and ranks these suggestions, achieving a recall rate of 53.4% among its top-5 recommendations, compared to 39.4% for previous tools. More recently, Chand et al. [11] present a comparative study of open-source LLMs for automating the *Extract Method* refactoring (e.g., multi-file changes) on Python code. They evaluate several resource-efficient models and show that iterative prompting strategies can improve both test pass rates and code quality metrics. This work provides insights into prompt engineering and model behavior for the extract method refactoring type.

In addition to single refactoring patterns, several recent works have explored LLM-driven refactoring from other dimensions. Liu et al. [38] provide one of the first empirical evaluations of general-purpose LLMs for automated refactoring, but their study is focused on single-file transformations and a small set of basic refactoring operations. Other studies investigate automated refactoring workflows that combine LLMs with IDE support or semantic retrieval: Next-Generation Refactoring for Extract Method [57] integrates LLM guidance with IDE-based program analysis for a single refactoring type, and LLM+IDE approaches for Move Method refactoring [5] focus on semantic embedding-based retrieval to support one specific refactoring type. Wu et al. [82] introduce iSMELL, a framework that combines LLMs with seven expert code smell detectors in order to improve code smell detection rather than large-scale refactoring generation. Their results show that iSMELL achieves an average F1 score of 75.17%, outperforming standalone LLMs in identifying code smells, and mention in future work that several of the detected smells can be mitigated using LLM-generated refactorings.

In contrast to prior work that focuses on specific refactoring types, constrained settings, or single-file transformations, our work provides a large-scale empirical study of the code refactoring capability of LLMs on real developer commit-level transformations. We quantify the impact of LLM-generated refactorings on architectural, implementation, testability, and design code smells, as well as code complexity, coupling, modularity, and cohesion. Moreover, we go beyond outcome-based metrics by analyzing the causes of failed unit tests in LLM-generated refactorings, categorizing failure cases such as external functionality changes, variable mismanagement, and incomplete modifications, which are factors that are largely overlooked in existing work.

6.4 Prompt Engineering for Improving LLMs

Prompt engineering is a technique to refine the input prompt provided to LLMs in order to enhance their performance. Previous studies [33, 61] have demonstrated that by carefully designing input prompts, the accuracy and quality of LLM-generated code can be significantly improved. Brown et al. [9] demonstrate that few-shot prompting techniques can guide LLMs to generate higher-quality code with less semantic and syntactic errors. Li et al. [35] propose a technique called Structured Chain-of-Thought (SCoT) to improve the performance of LLMs. SCoT prompting introduces programming structures (*i.e.,* sequential, branch, and loop) explicitly into the LLM's reasoning process, unlocking structured programming thinking in the model. Evaluations on code generation benchmarks such as HumanEval, MBPP, and MBCPP show that SCoT prompting outperforms CoT prompting by up to 13.79% in Pass@1, and human developers prefer the programs generated by SCoT prompting over those from CoT prompting. Shirafuji et al. [69] also explore the effect of few-shot prompting on LLM-based

refactoring. However, their evaluation is limited to standalone Python programs averaging 14.8 LOC and focuses only on reductions in cyclomatic complexity and LOC. Unlike our work, their study does not consider multi-file dependencies and does not evaluate a wide range of refactoring types or architectural-quality metrics.

Our study further investigates how modifications to input prompts, including zero-shot, one-shot, and chain-of-thought prompting, affect LLM refactoring performance on real-world commits.

7 Conclusion

In this study, we investigate the refactoring capabilities of LLMs and compare their performance. With the inclusion of newer, larger models such as GPT-4o and DeepSeek-v3, we find that these models outperform all others in improving static code quality metrics, while LLaMA 3 continues to deliver the strongest overall performance in reducing code smells (average 15.73%). Specifically, StarCoder2 demonstrates notable strength in addressing structural and readability-related code smells. DeepSeek-v3 and GPT-4o show the largest improvements in cohesion, coupling, and complexity, while GPT-4o Mini excels at resolving design and abstraction-related challenges. Developers remain most effective in performing complex, context-sensitive refactorings such as *Encapsulate Attribute*, highlighting areas where human expertise remains essential. Furthermore, we observe that prompt engineering significantly influences the quality of LLM-generated refactorings. Chain-of-thought prompting substantially enhances both functional correctness (unit test pass rates) and code smell reduction for StarCoder2 and GPT-4, underscoring the importance of structured reasoning and explicit instruction in guiding LLM performance. For LLaMA 3, the benefit from advanced prompting is limited, as it already achieves high performance under zero-shot conditions, indicating strong intrinsic reasoning capabilities. Overall, our study highlights the complementary strengths of human expertise and LLMs in refactoring tasks. Optimal results are likely achieved by integrating LLMs' automated systematic improvements with developer-driven context-aware modifications. Even though our results show only small differences between seen and unseen projects, future work should use stronger ways to rule out any impact from training-set overlap. One option is to build new or synthetic software projects that imitate real code but are guaranteed not to appear in any model's training data. Another is to test LLMs on private industrial projects, where the code was never part of public training corpora. Researchers can also use open-source LLMs with fully documented training sets, which makes it easier to check exactly where overlap might occur. In addition, future work should integrate dedicated refactoring purity checkers (e.g., Mantra) to more strictly separate refactoring-only commits from mixed changes and further strengthen the validity of large-scale evaluations. Future research should explore fine-tuning strategies and hybrid human-LLM workflows and assess the generalizability of these findings across diverse projects and programming languages.

References

[1] Roberta Arcoverde, Alessandro Garcia, and Eduardo Figueiredo. 2011. Understanding the longevity of code smells: preliminary results of an explanatory survey. In Proceedings of the 4th Workshop on Refactoring Tools. 33–36.

[2] Simone Balloccu, Patrícia Schmidtová, Mateusz Lango, and Ondrej Dusek. 2024. Leak, Cheat, Repeat: Data Contamination and Evaluation Malpractices in Closed-Source LLMs. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), Yvette Graham and Matthew Purver (Eds.). Association for Computational Linguistics, St. Julian's, Malta, 67–93. doi:10.18653/v1/2024.eacl-long.5

[3] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. 2016. DeepCoder: Learning to Write Programs. CoRR abs/1611.01989 (2016). arXiv:1611.01989 http://arxiv.org/abs/1611.01989

[4] Rajiv D. Banker, Srikant M. Datar, Chris F. Kemerer, and Dani Zweig. 1993. Software complexity and maintenance costs. Commun. ACM 36, 11 (Nov. 1993), 81–94. doi:10.1145/163359.163375

[5] Abhiram Bellur, Fraol Batole, Mohammed Raihan Ullah, Malinda Dilhara, Yaroslav Zharov, Timofey Bryksin, Kai Ishikawa, Haifeng Chen, Masaharu Morimoto, Shota Motoura, et al. 2025. Leveraging LLMs, IDEs, and Semantic Embeddings for Automated Move Method Refactoring. arXiv preprint arXiv:2503.20934 (2025).

[6] Berkeley Bootcamp. 2020. 11 Most In-Demand Programming Languages in 2021. [Online]. Available: https://bootcamp.berkeley.edu/blog/most-indemand-programming-languages/.
[7] Ana Carla Bibiano, Eduardo Fernandes, Daniel Oliveira, Alessandro Garcia, Marcos Kalinowski, Baldoino Fonseca, Roberto Oliveira, Anderson Oliveira, and Diego Cedrim. 2019. A quantitative study on characteristics and effect of batch refactoring on code smells. In 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). IEEE, 1–11.
[8] M. Bittman, R. Roos, and G.M. Kapfhammer. 2001. Creating a Free, Dependable Software Engineering Environment for Building Java Applications. In 1st Workshop on Open Source Software Engineering at ICSE 2001.
[9] Tom Brown et al. 2020. Language Models are Few-Shot Learners. In Advances in Neural Information Processing Systems, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf
[10] Diego Cedrim, Alessandro Garcia, Melina Mongiovi, Rohit Gheyi, Leonardo Sousa, Rafael de Mello, Baldoino Fonseca, Márcio Ribeiro, and Alexander Chávez. 2017. Understanding the impact of refactoring on smells: a longitudinal study of 23 software projects. In Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (Paderborn, Germany) (ESEC/FSE 2017). Association for Computing Machinery, New York, NY, USA, 465–475. doi:10.1145/3106237.3106259
[11] Sivajeet Chand, Melih Kilic, Roland Würsching, Sushant Kumar Pandey, and Alexander Pretschner. 2025. Automated Extract Method Refactoring with Open-Source LLMs: A Comparative Study. arXiv:2510.26480 [cs.SE] https://arxiv.org/abs/2510.26480
[12] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. ACM Transactions on Intelligent Systems and Technology 15, 3 (2024), 1–45.
[13] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code Generation with Generated Tests. arXiv:2207.10397 [cs.CL] https://arxiv.org/abs/2207.10397
[14] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] https://arxiv.org/abs/2107.03374
[15] Jinsu Choi, Gabin An, and Shin Yoo. 2024. Iterative Refactoring of Real-World Open-Source Programs with Large Language Models. In Proceedings of the 16th International Symposium on Search-Based Software Engineering. Lecture Notes in Computer Science, Vol. 14767. Springer Nature, 49–55.
[16] M. Claes and M. V. Mäntylä. 2020. 20-mad: 20 years of issues and commits of mozilla and apache development. In Proceedings of the 17th International Conference on Mining Software Repositories. 503–507.
[17] Don Coleman, Dan Ash, Bruce Lowther, and Paul Oman. 1994. Using metrics to evaluate software system maintainability. Computer 27, 8 (1994), 44–49.
[18] Jonathan Cordeiro, Shayan Noei, and Ying Zou. 2025. LLM-Driven Code Refactoring: Opportunities and Limitations. In 2025 IEEE/ACM Second IDE Workshop (IDE). IEEE, 32–36.
[19] DeepSeek-AI et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437
[20] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. IEEE Transactions on Software Engineering 50, 9 (Sept. 2024), 2254–2268. doi:10.1109/tse.2024.3428972
[21] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE). IEEE, 31–53.
[22] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated Repair of Programs from Large Language Models. In Proceedings of the 45th International Conference on Software Engineering (Melbourne, Victoria, Australia) (ICSE '23). IEEE Press, 1469–1481. doi:10.1109/ICSE48619.2023.00128
[23] Martin Fowler. 2018. Refactoring: improving the design of existing code. Addison-Wesley Professional.
[24] M. Fowler, K. Beck, J. Brant, W. Opdyke, and D. Roberts. 1999. Refactoring: Improving the Design of Existing Code. Addison-Wesley Professional, Berkeley, CA, USA.
[25] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic test suite generation for object-oriented software. SIGSOFT/FSE 2011 - Proceedings of the 19th ACM SIGSOFT Symposium on Foundations of Software Engineering, 416–419. doi:10.1145/2025113.2025179
[26] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783
[27] Refactoring Guru. 2024. https://refactoring.guru/refactoring/techniques
[28] John A Hartigan and Manchek A Wong. 1979. Algorithm AS 136: A k-means clustering algorithm. Journal of the royal statistical society. series c (applied statistics) 28, 1 (1979), 100–108.
[29] Paul Jansen. 2024. https://www.tiobe.com/tiobe-index/
[30] JetBrains. 2025. IntelliJ IDEA: The Java IDE for Professional Developers. https://www.jetbrains.com/idea Version 2024.1, JetBrains s.r.o..
[31] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data 7, 3 (2019), 535–547.
[32] M. Kim et al. 2021. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing.

[33] Takeshi Kojima, Shixiang (Shane) Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. In Advances in Neural Information Processing Systems, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 22199–22213. https://proceedings.neurips.cc/paper_files/paper/2022/file/8bb0d291acd4acf06ef112099c16f326-Paper-Conference.pdf
[34] Mira Leung and Gail Murphy. 2023. On Automated Assistants for Software Development: The Role of LLMs. In 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). 1737–1741. doi:10.1109/ASE56229.2023.00035
[35] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023. Structured chain-of-thought prompting for code generation. ACM Transactions on Software Engineering and Methodology (2023).
[36] Raymond Li et al. 2023. StarCoder: may the source be with you! arXiv:2305.06161 [cs.CL] https://arxiv.org/abs/2305.06161
[37] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. Science 378, 6624 (2022), 1092–1097.
[38] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2025. Exploring the potential of general purpose LLMs in automated software refactoring: an empirical study. Automated Software Engineering 32, 1 (2025), 26.
[39] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. 2024. Exploring and evaluating hallucinations in llm-powered code generation. arXiv preprint arXiv:2404.00971 (2024).
[40] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172 [cs.CL] https://arxiv.org/abs/2307.03172
[41] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. arXiv preprint arXiv:2402.19173 (2024).
[42] Linghui Luo, Martin Schäf, Daniel Sanchez, and Eric Bodden. 2021. IDE support for cloud-based static analyses. In Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021). Association for Computing Machinery, New York, NY, USA, 1178–1189. doi:10.1145/3468264.3468535
[43] Thomas J McCabe. 1976. A complexity measure. IEEE Transactions on software Engineering 4 (1976), 308–320.
[44] T. Mens and T. Tourwe. 2004. A survey of software refactoring. IEEE Transactions on Software Engineering 30, 2 (2004), 126–139. doi:10.1109/TSE.2004.1265817
[45] Frederic P Miller, Agnes F Vandome, and John McBrewster. 2010. Apache Maven. Alpha Press.
[46] Emerson Murphy-Hill, Chris Parnin, and Andrew P. Black. 2012. How We Refactor, and How We Know It. IEEE Transactions on Software Engineering 38, 1 (2012), 5–18. doi:10.1109/TSE.2011.41
[47] Kumiyo Nakakoji, Yasuhiro Yamamoto, Yoshiyuki Nishinaka, Kouichi Kishida, and Yunwen Ye. 2002. Evolution patterns of open-source software systems and communities. In Proceedings of the International Workshop on Principles of Software Evolution (Orlando, Florida) (IWPSE '02). Association for Computing Machinery, New York, NY, USA, 76–85. doi:10.1145/512035.512055
[48] Shayan Noei, Heng Li, Stefanos Georgiou, and Ying Zou. 2023. An Empirical Study of Refactoring Rhythms and Tactics in the Software Development Process. IEEE Transactions on Software Engineering 01 (2023), 1–17.
[49] Shayan Noei, Heng Li, and Ying Zhou. 2025. An Empirical Study on Release-Wise Refactoring Patterns. Proceedings of the ACM on Software Engineering 2, FSE (2025).
[50] Alberto S Nuñez-Varela, Héctor G Pérez-Gonzalez, Francisco E Martínez-Perez, and Carlos Soubervielle-Montalvo. 2017. Source code metrics: A systematic mapping study. Journal of Systems and Software 128 (2017), 164–197.
[51] William Opdyke and Ralph Johnson. 1992. Refactoring Object-Oriented Frameworks. (07 1992).
[52] OpenAI et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774
[53] Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Fausto Fasano, Rocco Oliveto, and Andrea De Lucia. 2018. On the diffuseness and the impact on maintainability of code smells: a large scale empirical investigation. In Proceedings of the 40th International Conference on Software Engineering (Gothenburg, Sweden) (ICSE '18). Association for Computing Machinery, New York, NY, USA, 482. doi:10.1145/3180155.3182532
[54] Sheena Panthaplackel, Pengyu Nie, Milos Gligoric, Junyi Jessy Li, and Raymond Mooney. 2020. Learning to Update Natural Language Comments Based on Code Changes. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 1853–1868. doi:10.18653/v1/2020.acl-main.168
[55] Jevgenija Pantiuchina, Fiorella Zampetti, Simone Scalabrino, Valentina Piantadosi, Rocco Oliveto, Gabriele Bavota, and Massimiliano Di Penta. 2020. Why Developers Refactor Source Code: A Mining-based Study. ACM Trans. Softw. Eng. Methodol. 29, 4, Article 29 (sep 2020), 30 pages. doi:10.1145/3408302
[56] David Lorge Parnas. 1972. On the criteria to be used in decomposing systems into modules. Commun. ACM 15, 12 (1972), 1053–1058.
[57] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Timofey Bryksin, and Danny Dig. 2024. Next-generation refactoring: Combining llm insights and ide capabilities for extract method. In 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 275–287.

[58] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Andrey Sokolov, Timofey Bryksin, and Danny Dig. 2024. EM-Assist: Safe Automated ExtractMethod Refactoring with LLMs. In Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE '24). ACM, 582–586. doi:10.1145/3663529.3663803

[59] Mohammad Masudur Rahman and Chanchal K. Roy. 2014. An insight into the pull requests of GitHub. In Proceedings of the 11th Working Conference on Mining Software Repositories (Hyderabad, India) (MSR 2014). Association for Computing Machinery, New York, NY, USA, 364–367. doi:10.1145/2597073.2597121

[60] Peter J Rousseeuw. 1987. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of computational and applied mathematics 20 (1987), 53–65.

[61] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2025. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. arXiv:2402.07927 [cs.AI] https://arxiv.org/abs/2402.07927

[62] Scientific Toolworks, Inc. 2023. Understand. https://scitools.com/static-code-analysis Software for Static Code Analysis.

[63] scitools. 2024. https://documentation.scitools.com/pdf/metricsdoc.pdf

[64] Ketan Rajshekhar Shahapure and Charles Nicholas. 2020. Cluster quality analysis using silhouette score. In 2020 IEEE 7th international conference on data science and advanced analytics (DSAA). IEEE, 747–748.

[65] Yunfan Shao, Linyang Li, Zhaoye Fei, Hang Yan, Dahua Lin, and Xipeng Qiu. 2024. Balanced Data Sampling for Language Model Training with Clustering. arXiv:2402.14526 [cs.CL] https://arxiv.org/abs/2402.14526

[66] Tushar Sharma. 2024. Multi-faceted Code Smell Detection at Scale using DesigniteJava 2.0. In Proceedings of the 21st International Conference on Mining Software Repositories (Lisbon, Portugal) (MSR '24). Association for Computing Machinery, New York, NY, USA, 284–288. doi:10.1145/3643991.3644881

[67] T. Sharma, P. Mishra, and R. Tiwari. 2016. Designite: A software design quality assessment tool. In Proceedings of the 1st International Workshop on Bringing Architectural Design Thinking into Developers' Daily Activities. 1–4.

[68] Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, Yuenan Guo, and Qianxiang Wang. 2023. PanGu-Coder2: Boosting Large Language Models for Code with Ranking Feedback. arXiv:2307.14936 [cs.CL] https://arxiv.org/abs/2307.14936

[69] Atsushi Shirafuji, Yusuke Oda, Jun Suzuki, Makoto Morishita, and Yutaka Watanobe. 2023. Refactoring Programs Using Large Language Models with Few-Shot Examples. In 2023 30th Asia-Pacific Software Engineering Conference (APSEC). 151–160. doi:10.1109/APSEC60848.2023.00025

[70] Danilo Silva, Nikolaos Tsantalis, and Marco Tulio Valente. 2016. Why we refactor? confessions of GitHub contributors. In Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (Seattle, WA, USA) (FSE 2016). Association for Computing Machinery, New York, NY, USA, 858–870. doi:10.1145/2950290.2950305

[71] David Steinberg, Frank Budinsky, Marcelo Paternostro, and Ed Merks. 2009. EMF: Eclipse Modeling Framework 2.0 (2nd ed.). Addison-Wesley Professional.

[72] Wayne P. Stevens, Glenford J. Myers, and Larry L. Constantine. 1974. Structured design. IBM systems journal 13, 2 (1974), 115–139.

[73] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2017. An Empirical Comparison of Model Validation Techniques for Defect Prediction Models. 1 (2017).

[74] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2018. The Impact of Automated Parameter Optimization for Defect Prediction Models. (2018).

[75] Wei Tao, Yanlin Wang, Ensheng Shi, Lun Du, Shi Han, Hongyu Zhang, Dongmei Zhang, and Wenqiang Zhang. 2021. On the evaluation of commit message generation models: An experimental study. In 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 126–136.

[76] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] https://arxiv.org/abs/2302.13971

[77] Nikolaos Tsantalis, Ameya Ketkar, and Danny Dig. 2020. RefactoringMiner 2.0. IEEE Transactions on Software Engineering 48, 3 (2020), 930–950.

[78] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and Efficient Refactoring Detection in Commit History. In Proceedings of the 40th International Conference on Software Engineering (Gothenburg, Sweden) (ICSE '18). ACM, New York, NY, USA, 483–494. doi:10.1145/3180155.3180206

[79] Eva Van Emden and Leon Moonen. 2002. Java quality assurance by detecting code smells. In Ninth Working Conference on Reverse Engineering, 2002. Proceedings. IEEE, 97–106.

[80] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. arXiv:2002.10957 [cs.CL] https://arxiv.org/abs/2002.10957

[81] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 172–184.

[82] Di Wu, Fangwen Mu, Lin Shi, Zhaoqiang Guo, Kui Liu, Weiguang Zhuang, Yuqi Zhong, and Li Zhang. 2024. iSMELL: Assembling LLMs with Expert Toolsets for Code Smell Detection and Refactoring. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1345–1357. doi:10.1145/3691620.3695508

[83] Cong Xu, Gayathri Saranathan, Mahammad Parwez Alam, Arpit Shah, James Lim, Soon Yee Wong, Foltin Martin, and Suparna Bhattacharya. 2024. Data Efficient Evaluation of Large Language Models and Text-to-Image Models via Adaptive Sampling. arXiv:2406.15527 [cs.LG] https://arxiv.org/abs/2406.15527

[84] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (San Diego, CA, USA) (MAPS 2022). Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/3520312.3534862

[85] Frank F. Xu, Bogdan Vasilescu, and Graham Neubig. 2022. In-IDE Code Generation from Natural Language: Promise and Challenges. ACM Trans. Softw. Eng. Methodol. 31, 2, Article 29 (mar 2022), 47 pages. doi:10.1145/3487569

[86] Yisen Xu, Feng Lin, Jinqiu Yang, Nikolaos Tsantalis, et al. 2025. Mantra: Enhancing automated method-level refactoring with contextual RAG and multi-agent LLM collaboration. arXiv preprint arXiv:2503.14340 (2025).

[87] Norihiro Yoshida, Tsubasa Saika, Eunjong Choi, Ali Ouni, and Katsuro Inoue. 2016. Revisiting the relationship between code smells and refactoring. In 2016 IEEE 24th International Conference on Program Comprehension (ICPC). IEEE, 1–4.

[88] Edward Yourdon and Larry L Constantine. 1979. Structured design: fundamentals of a discipline of computer program and systems design. Prentice-Hall, Inc.

[89] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pre-trained Models. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24). Association for Computing Machinery, New York, NY, USA, Article 37, 12 pages. doi:10.1145/3597503.3623316

[90] J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts. In Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (, Hamburg, Germany,) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 437, 21 pages. doi:10.1145/3544548.3581388

[91] Jian Zhou, Hongyu Zhang, and David Lo. 2012. Where should the bugs be fixed? more accurate information retrieval-based bug localization based on bug reports. In 2012 34th International conference on software engineering (ICSE). IEEE, 14–24.

[92] Zhichao Zhou, Yutian Tang, Yun Lin, and Jingzhu He. 2024. An LLM-based Readability Measurement for Unit Tests' Context-aware Inputs. arXiv:2407.21369 [cs.SE] https://arxiv.org/abs/2407.21369